UNIVERSITÀ DI PARMA

Camera Models (2)

Summary

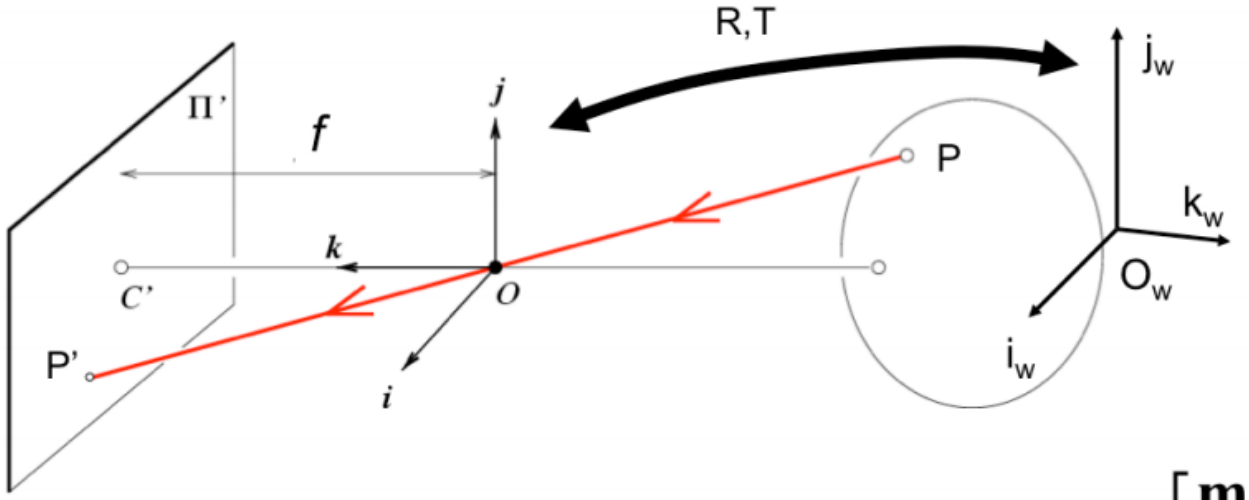
UNIVERSITÀ DI PARMA

- Pin Hole Camera recap
- Calibration

UNIVERSITÀ DI PARMA

Pin Hole Model recap

Perspective Transformation



$$P'_{3\times1} = M\,P_w = K_{3\times3}\begin{bmatrix} R & T \end{bmatrix}_{3\times4} P_{w\,4\times1} \qquad M = \begin{bmatrix} \mathbf{m}_1 \\ \mathbf{m}_2 \\ \mathbf{m}_3 \end{bmatrix}$$

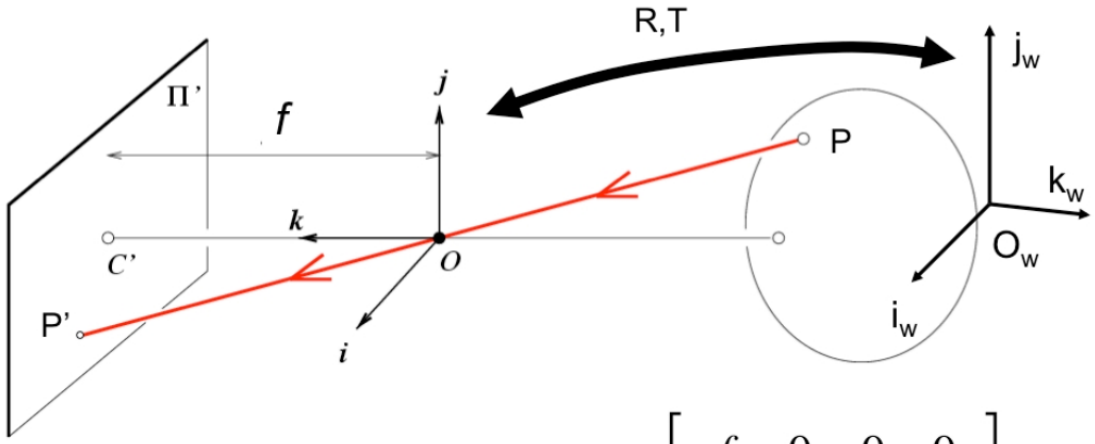
$$= \begin{bmatrix} \mathbf{m}_1 \\ \mathbf{m}_2 \\ \mathbf{m}_3 \end{bmatrix} P_W = \begin{bmatrix} \mathbf{m}_1 P_W \\ \mathbf{m}_2 P_W \\ \mathbf{m}_3 P_W \end{bmatrix}$$

$$\mathbf{E} \rightarrow \left(\frac{\mathbf{m}_1 P_w}{\mathbf{m}_3 P_w}, \frac{\mathbf{m}_2 P_w}{\mathbf{m}_3 P_w}\right) \quad \text{[Eq.12]}$$

Courtesy: Silvio Savarese

Perspective Transformation (ideal case)

- Simplified situation: no rotation, no translation, no skew, no offset, squared pixels



$$M = K\begin{bmatrix} R & T \end{bmatrix} = K\begin{bmatrix} I & 0 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\rightarrow P'_E = \left(\frac{\mathbf{m}_1 P_w}{\mathbf{m}_3 P_w}, \frac{\mathbf{m}_2 P_w}{\mathbf{m}_3 P_w}\right) = \left(f\frac{x_w}{z_w}, f\frac{y_w}{z_w}\right) \qquad P_w = \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Courtesy: Silvio Savarese

Perspective Transformation

- 11 degrees of freedom

$$P' = M P_w = K[R \quad T] P_w$$

Internal parameters

External parameters

$$\mathcal{M} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix}_{3\times 4}$$

$$K = \begin{bmatrix} \alpha & -\alpha\cot\theta & u_o \\ 0 & \frac{\beta}{\sin\theta} & v_o \\ 0 & 0 & 1 \end{bmatrix} \qquad R = \begin{bmatrix} \mathbf{r}_1^T \\ \mathbf{r}_2^T \\ \mathbf{r}_3^T \end{bmatrix} \qquad T = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

Calibration
Tsai approach (1987)

Calibration

$$P' = M P_w = K[R \quad T] P_w$$

Internal parameters

External parameters

- Calibration ⟶ **estimation of intrinsic/extrinsic parameters**
- We need 1 or, rather, more images

Change notation:
$P = P_w$

$p = P'$

Calibration

UNIVERSITÀ DI PARMA

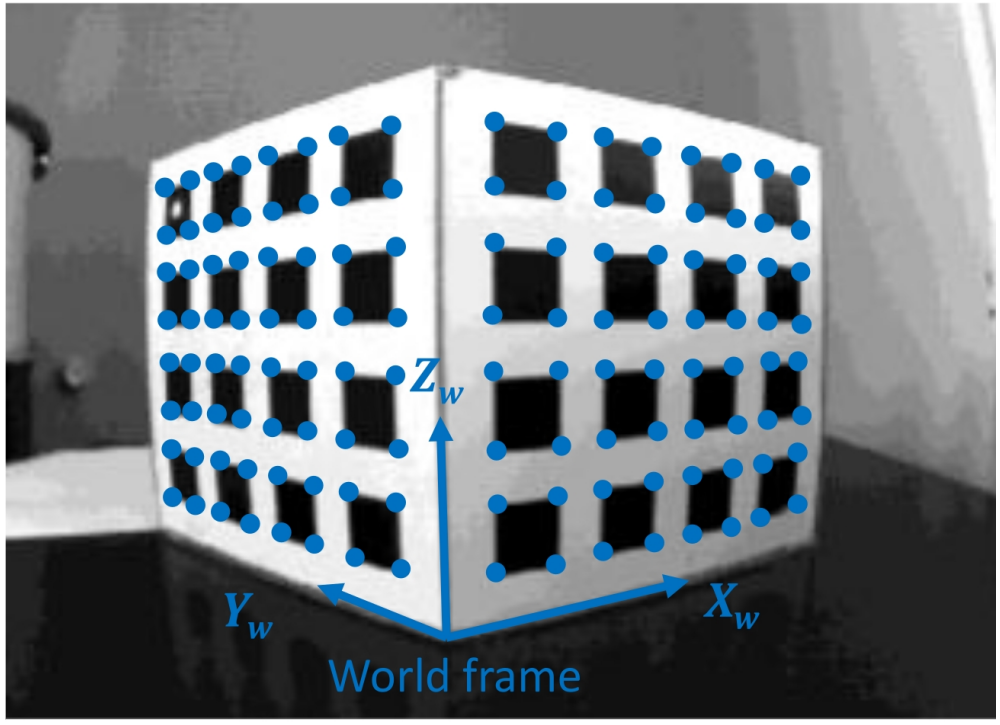
- Small notation change
  - $P_W \longrightarrow P$
  - $P' \longrightarrow p$

$$p = K \left[R \, T \right] P = MP$$

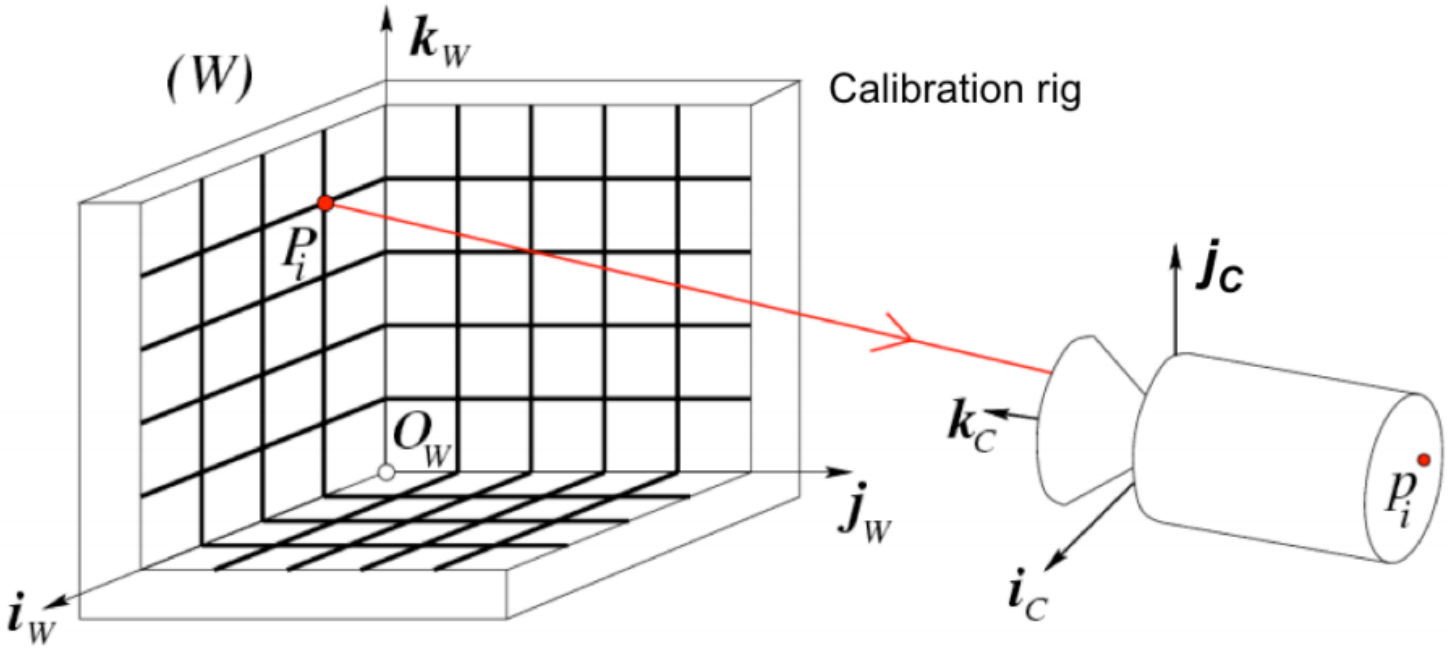
Tsai's method

UNIVERSITÀ DI PARMA

- We use 3D calibration targets and their projection on an image



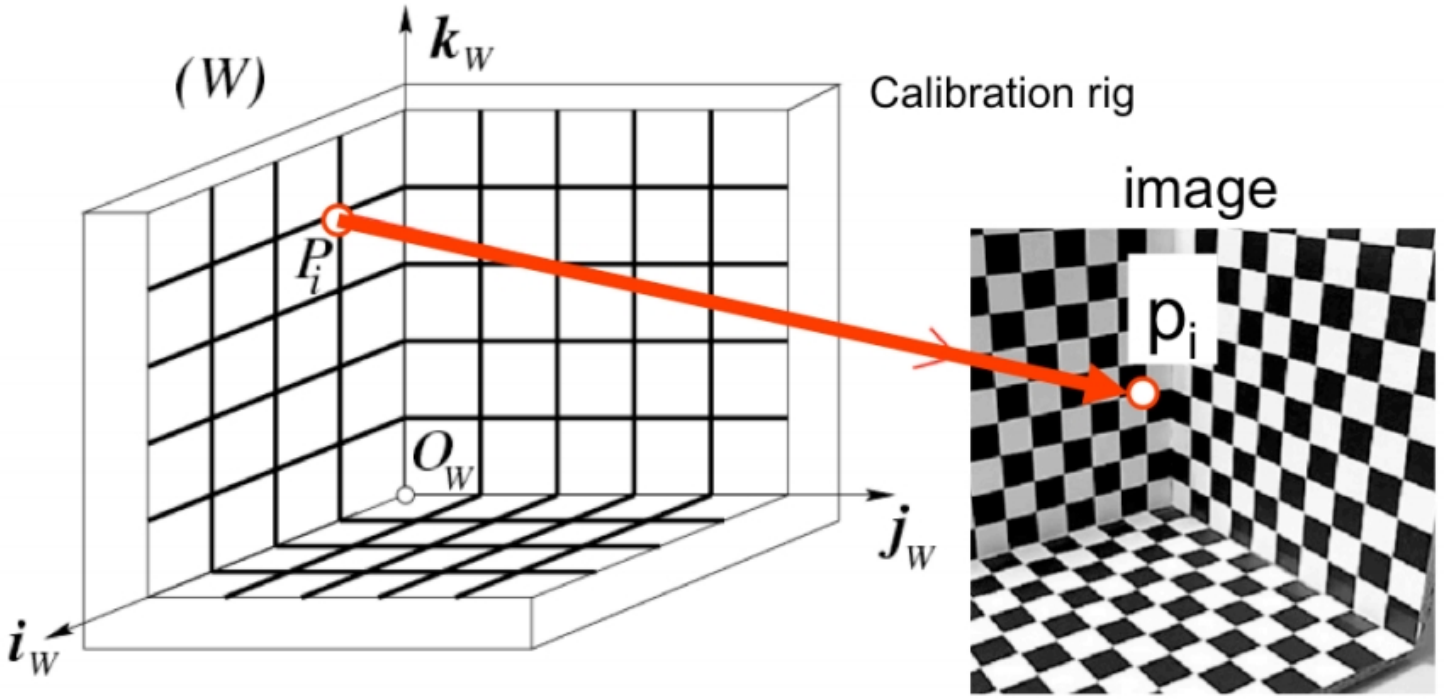
Calibration



- We need: n points (P_1, P_2,…,P_n) in known positions [O_w,i_w,j_w,k_w] in the world reference system

Courtesy: Silvio Savarese

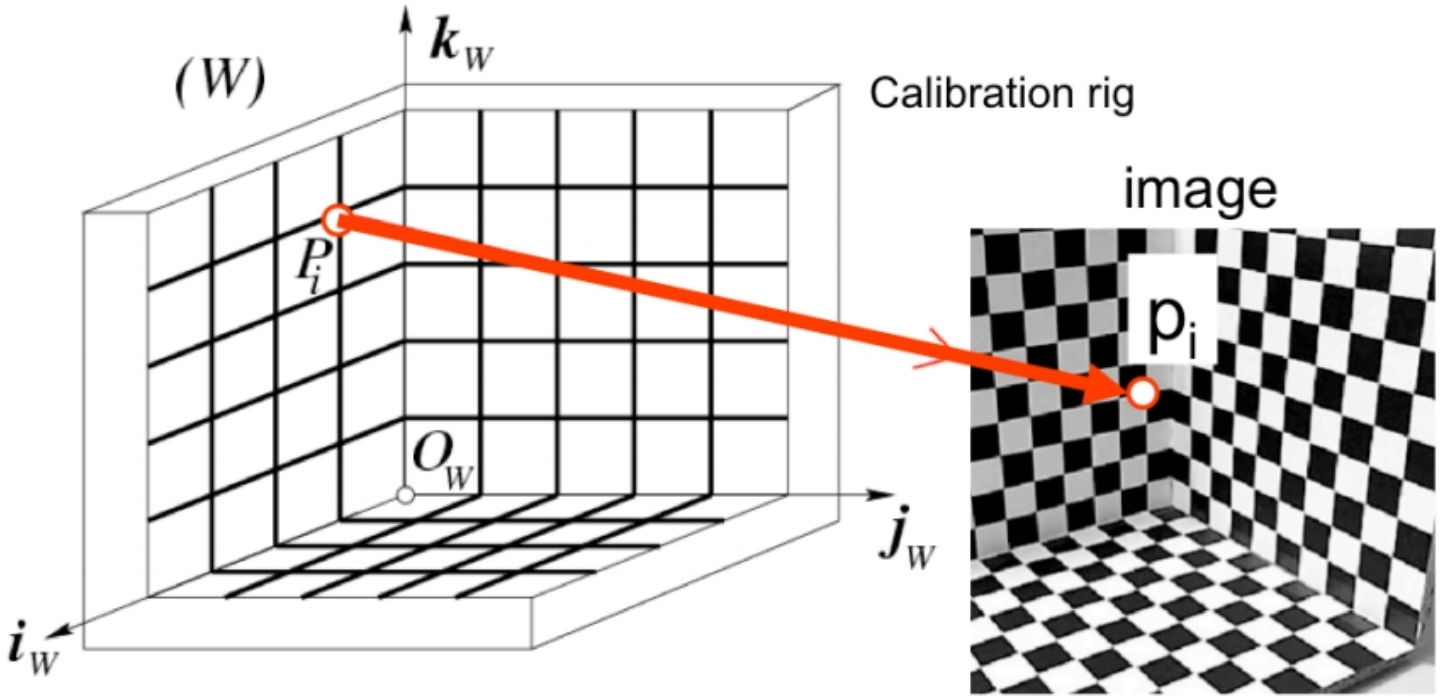
Calibration



- We also need: camera coordinates for the n points projections (p_1, p_2,…,p_n) as (i_c,j_c)

Courtesy: Silvio Savarese

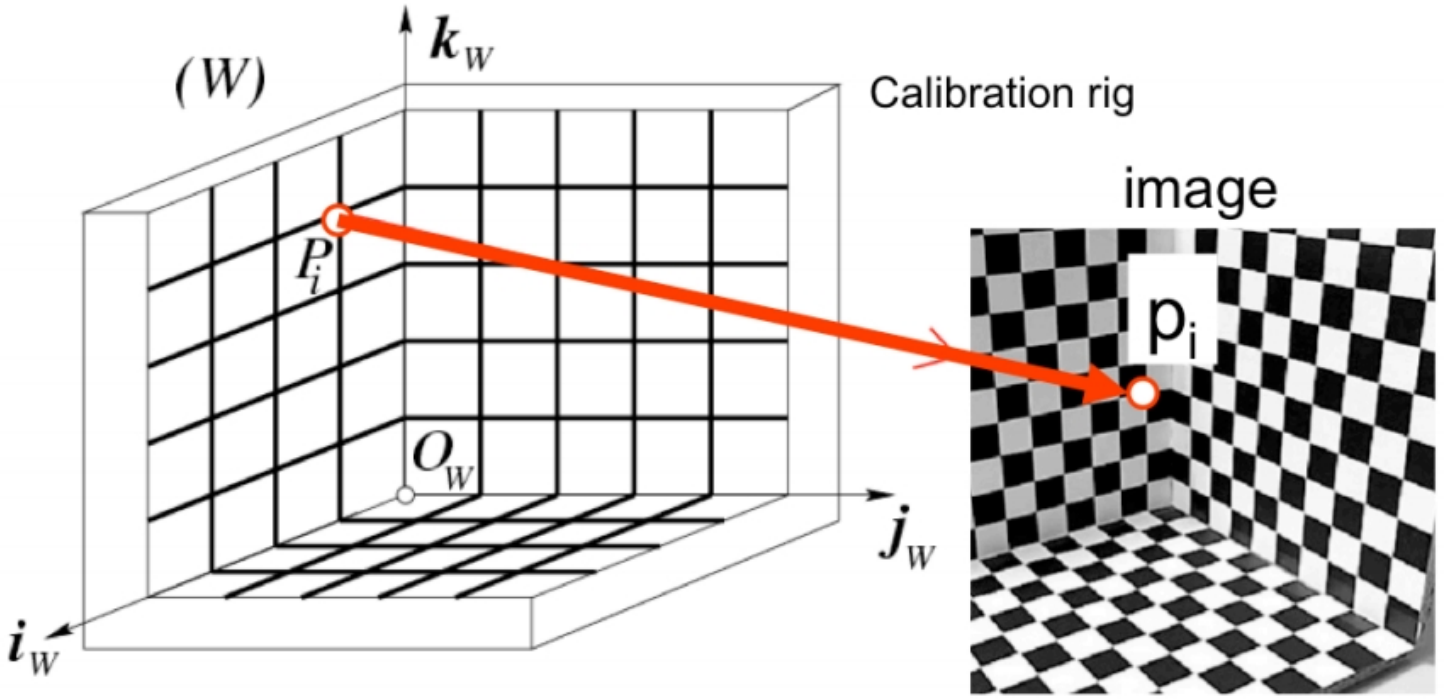
Calibration



- 11 degrees of freedom → how many matches we need?
- Each correspondence → 2 equations (world coordinates → row/column)
- We need, at least, 6 correspondences

Courtesy: Silvio Savarese

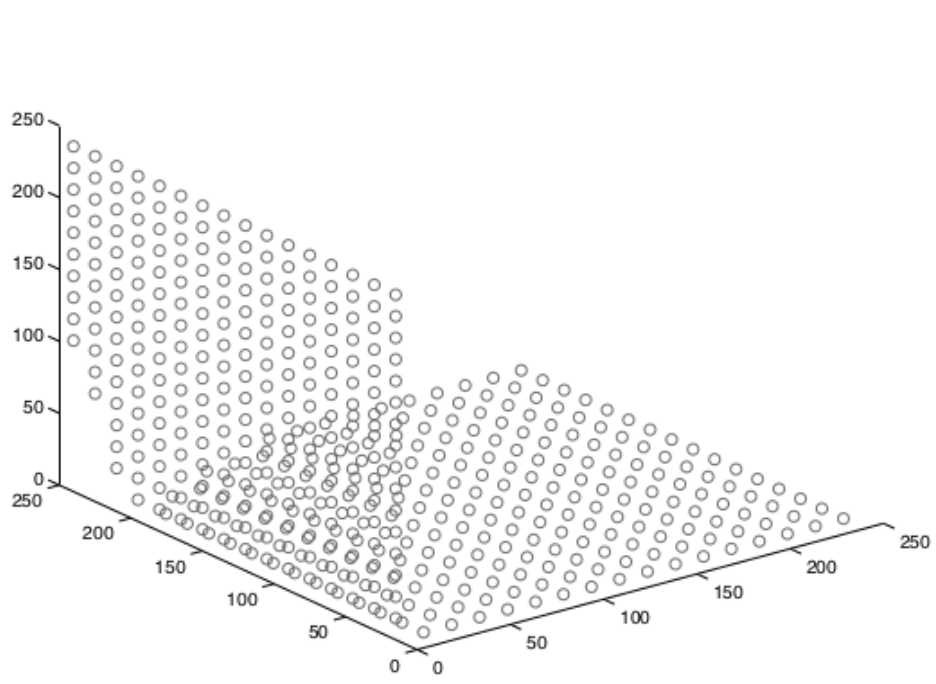
Calibration

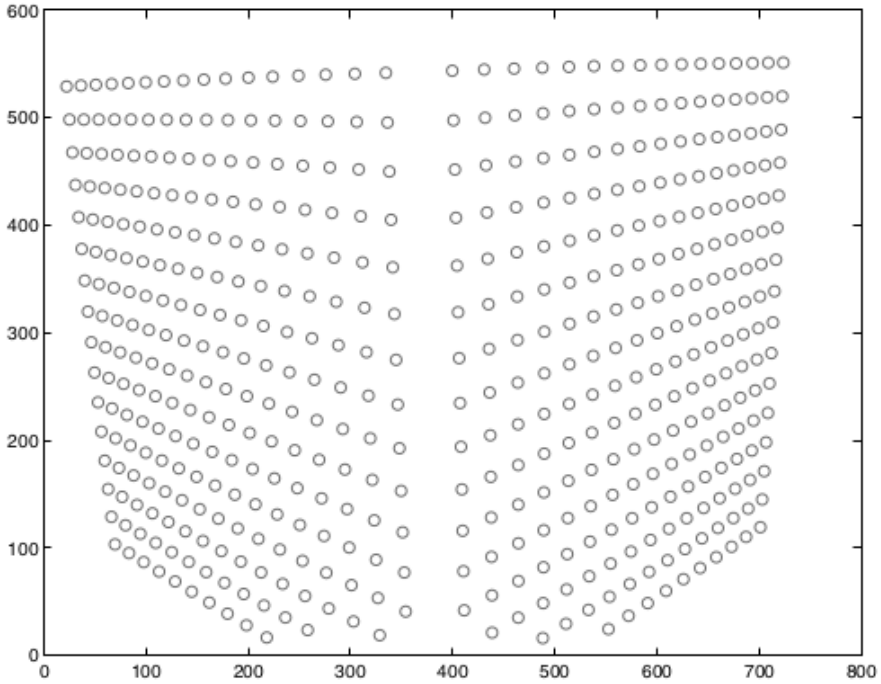


- Practically we need more than 6 correspondences!
- Consider error in measurements + digitalization error!

Courtesy: Silvio Savarese

Example of calibration data





- Example of 491 3D points
  - From: Janne Heikkilä; data copyright 2000, University of Oulu.

DLT Algorithm

UNIVERSITÀ DI PARMA

- 12 elements but actually there is a dependence among them → 11 unknowns

$$p = K \left[R \, T \right] P = MP$$

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

DLT Algorithm

UNIVERSITÀ DI PARMA

- Direct Linear Transform: rewrite the perspective projection equation as homogeneous system of linear equations

$$p = K \left[R \, T \right] P = MP$$

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

Calibration

UNIVERSITÀ DI PARMA

$$u_i = \frac{m_{00} X_i + m_{01} Y_i + m_{02} Z_i + m_{03}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}} \qquad v_i = \frac{m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}}$$

$$m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23} - m_{00} u_i X_i - m_{01} u_i Y_i - m_{02} u_i Z_i - m_{03} u_i = 0$$

$$m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13} - m_{20} v_i X_i - m_{21} v_i Y_i - m_{22} v_i Z_i - m_{23} v_i = 0$$

We can recover 2 equations for each correspondence world ↔ image

Calibration

UNIVERSITÀ DI PARMA

Homogeneous System

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 & -v_1 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n & -v_n \end{bmatrix} \begin{matrix} m_{00} \\ m_{01} \\ m_{02} \\ m_{03} \\ m_{10} \\ m_{11} \\ m_{12} \\ m_{13} \\ m_{20} \\ m_{21} \\ m_{22} \\ m_{23} \end{matrix} = \begin{matrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{matrix}$$

Calibration

UNIVERSITÀ DI PARMA

- Step back and write in a simpler form

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_0 \\ m_1 \\ m_2 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_0 \\ m_1 \\ m_2 \end{bmatrix} P_i$$

Calibration

UNIVERSITÀ DI PARMA

- Step back and write in a simpler form

$$u_i = \frac{m_{00} X_i + m_{01} Y_i + m_{02} Z_i + m_{03}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}} \qquad v_i = \frac{m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}}$$

$$u_i = \frac{m_0 P_i}{m_2 P_i} \qquad v_i = \frac{m_1 P_i}{m_2 P_i}$$

$$-m_0 P_i + u_i (m_2 P_i) = 0 \qquad -m_1 P_i + v_i (m_2 P_i) = 0$$

Calibration

UNIVERSITÀ DI PARMA

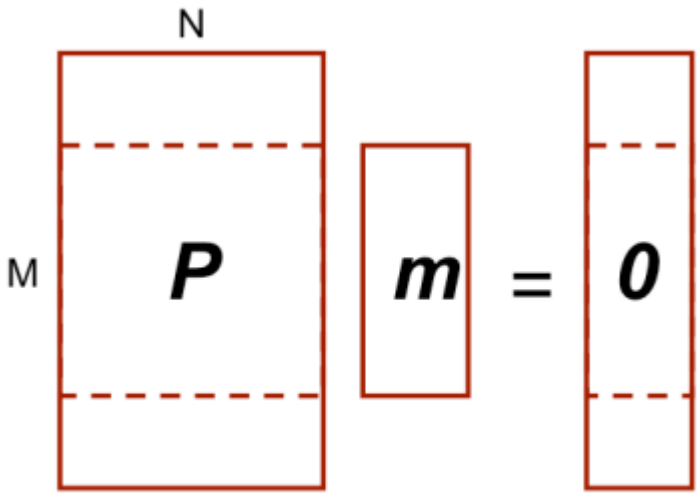
$$-m_0 P_i + u_i (m_2 P_i) = 0 \qquad -m_1 P_i + v_i (m_2 P_i) = 0$$

$$\begin{bmatrix} -P_0^T & 0^T & u_0 P_0^T \\ 0^T & -P_0^T & v_0 P_0^T \\ \dots & \dots & \dots \\ -P_n^T & 0^T & u_n P_n^T \\ 0^T & -P_n^T & v_n P_n^T \end{bmatrix}_{2n \times 12} \begin{bmatrix} m_0^T \\ m_1^T \\ m_2^T \end{bmatrix}_{12 \times 1} = \begin{bmatrix} 0 \\ \dots \\ 0 \end{bmatrix} \Rightarrow Pm = 0$$

Homogeneous System → 2n equations

Unknowns → m_i

Calibration



- M>N as we suggested
  - Rectangular system of equations
  - Overdetermined
    - "potentially" no solution other trivial one m=0
- The idea is to find the best fitting solution
- Minimize $|\mathrm{P}m|^2$
  - Assuming m != 0 $\longrightarrow |m|^2 = 1$

Courtesy: Silvio Savarese

Recap: inverting a non-square matrix

UNIVERSITÀ DI PARMA

- A linear equation

$$Ax = y$$

- Can be solved finding a matrix B (ideally $B=A^{-1}$, namely $AB=I$)

$$x = By$$

- When A is taller than it is wide → possibly no solutions
- When A is wider than it is tall → multiple solutions

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- Partially generalize matrix inversion for non square matrices
  - Do you remember something?
- Each matrix can be factorized into **singular vectors** and **singular values**
- More precisely each matrix A can be seen as product of three matrices:

$$A = UDV^T$$

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- Supposing A as $m \times n$ matrix, U is a $m \times m$ matrix, D is a $m \times n$ one and V is a $n \times n$ one
- U and D orthogonal matrices
  - Namely $UU^T=U^TU=I$ and $VV^T=V^TV=I$
  - Columns are the left (U) or right (V) singular vectors
- D is a diagonal matrix
  - Elements along the diagonal are the singular values

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- A linear equation

$$Ax = y$$

- Can be solved finding a matrix B (ideally $B=A^{-1}$)

$$x = By$$

- When A is taller than it is wide → possibly no solutions
- When A is wider than it is tall → multiple solutions

Moore-Penrose pseudoinverse

UNIVERSITÀ DI PARMA

- The pseudoinverse of a matrix A or A^+ is defined as

$$A^+ = \lim_{\alpha \to 0} \left(A^T A + \alpha I \right)^{-1} A^T$$

- Practically A^+ can be approximated using SVD

$$A^+ = V D^+ U^T$$

- When A is taller than it is wide this allows to compute the best approximation

Calibration

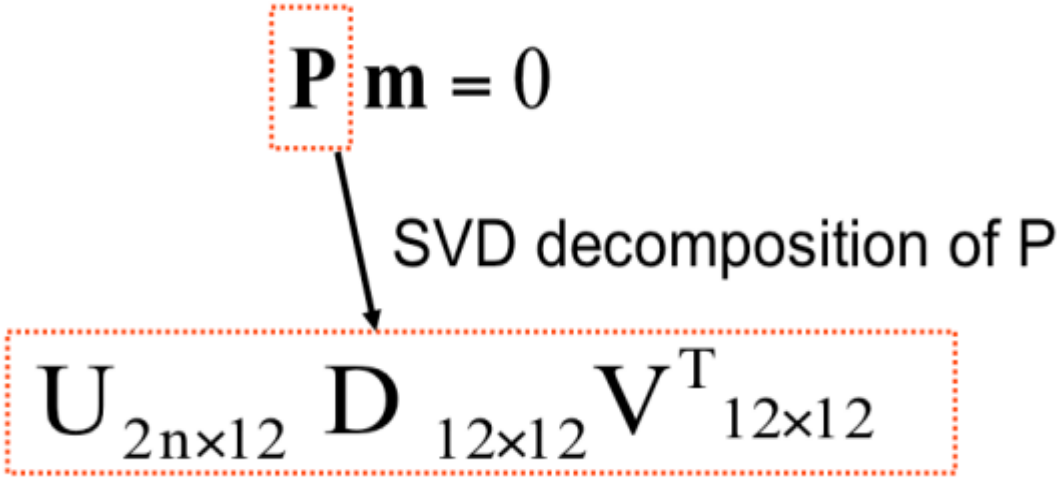
UNIVERSITÀ DI PARMA

- How we can solve such system?

$$\mathrm{P}\,m=0$$

- Via SVD decomposition!
  - This leads to the optimal solution assuming $|m|^2=1$

Calibration



- The last V column gives us m

$$\boldsymbol{m} \overset{\text{def}}{=} \begin{pmatrix} \mathbf{m}_1^T \\ \mathbf{m}_2^T \\ \mathbf{m}_3^T \end{pmatrix} \quad \Rightarrow \quad M$$

Courtesy: Silvio Savarese

Parameters Extraction

UNIVERSITÀ DI PARMA

Courtesy: Silvio Savarese

$$M = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} \rho$$

- We can recover parameters but with a scale factor ρ
- Actually not an unknown, we can impose $|m|=1$
  - or equivalent Froebius norm $||M||=1$
- Extrinsic and Intrinsic parameters are separately extracted

Parameters Extraction

$$\frac{M}{\rho} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} = \frac{K}{\rho} \begin{bmatrix} R & T \end{bmatrix}$$

$$K = \begin{bmatrix} \alpha & -\alpha\cot\theta & u_o \\ 0 & \frac{\beta}{\sin\theta} & v_o \\ 0 & 0 & 1 \end{bmatrix}$$

$A_{3\times3}$ $\quad$ $\mathbf{b}_{1\times3}$

Box 1

$$\mathrm{A} = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \mathbf{a}_3^T \end{bmatrix} \qquad \mathrm{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Estimated values

Intrinsic

$$\rho = \frac{\pm 1}{|\mathbf{a}_3|} \qquad u_o = \rho^2 (\mathbf{a}_1 \cdot \mathbf{a}_3)$$

$$\mathrm{v_o} = \rho^2 (\mathbf{a}_2 \cdot \mathbf{a}_3)$$

$$\cos\theta = \frac{(\mathbf{a}_1 \times \mathbf{a}_3) \cdot (\mathbf{a}_2 \times \mathbf{a}_3)}{|\mathbf{a}_1 \times \mathbf{a}_3| \cdot |\mathbf{a}_2 \times \mathbf{a}_3|}$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Parameters Extraction

$$\frac{\mathcal{M}}{\rho} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} = \mathrm{K}[\mathrm{R} \quad \mathrm{T}]$$

A b

$$\mathrm{A} = \begin{bmatrix} \mathbf{a}_1^{\mathrm{T}} \\ \mathbf{a}_2^{\mathrm{T}} \\ \mathbf{a}_3^{\mathrm{T}} \end{bmatrix} \quad \mathrm{b} = \begin{bmatrix} \mathrm{b}_1 \\ \mathrm{b}_2 \\ \mathrm{b}_3 \end{bmatrix}$$

Estimated values

Intrinsic

$$\alpha = \rho^2 |\mathbf{a}_1 \times \mathbf{a}_3| \sin\theta$$

$$\beta = \rho^2 |\mathbf{a}_2 \times \mathbf{a}_3| \sin\theta$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Parameters Extraction

UNIVERSITÀ DI PARMA

$$\frac{\mathcal{M}}{\rho} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} = \mathrm{K}[\mathrm{R} \quad \mathrm{T}]$$

A b

$$\mathrm{A} = \begin{bmatrix} \mathbf{a}_1^{\mathrm{T}} \\ \mathbf{a}_2^{\mathrm{T}} \\ \mathbf{a}_3^{\mathrm{T}} \end{bmatrix} \qquad \mathrm{b} = \begin{bmatrix} \mathrm{b}_1 \\ \mathrm{b}_2 \\ \mathrm{b}_3 \end{bmatrix}$$

Estimated values

Extrinsic

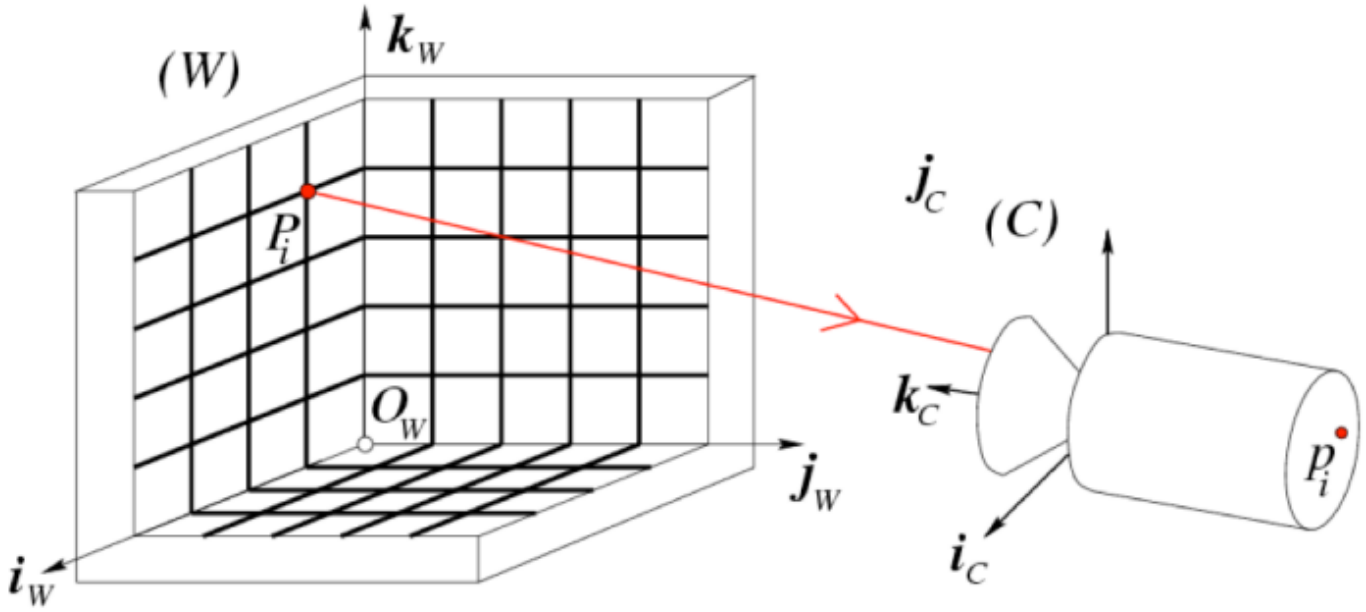
$$\mathbf{r}_1 = \frac{(\mathbf{a}_2 \times \mathbf{a}_3)}{|\mathbf{a}_2 \times \mathbf{a}_3|} \qquad \mathbf{r}_3 = \frac{\pm \mathbf{a}_3}{|\mathbf{a}_3|}$$

$$\mathbf{r}_2 = \mathbf{r}_3 \times \mathbf{r}_1 \qquad \mathrm{T} = \rho\, \mathrm{K}^{-1}\mathbf{b}$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Degenerate Case



- Not all P_ws lead to a result!
  - P_w points must not lie on the same plane
  - P_w points must not lie on the intersection curve of 2 quadric surfaces

Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

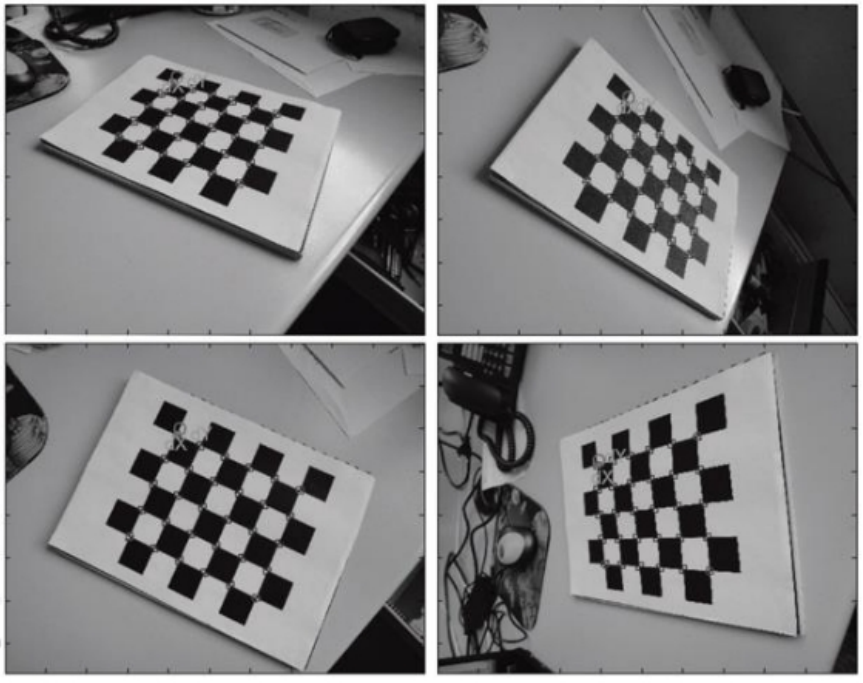
Calibration
Zhang approach (2000)

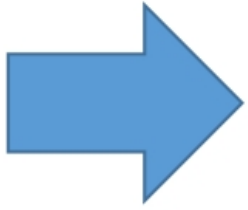
Zhang approach (2000)

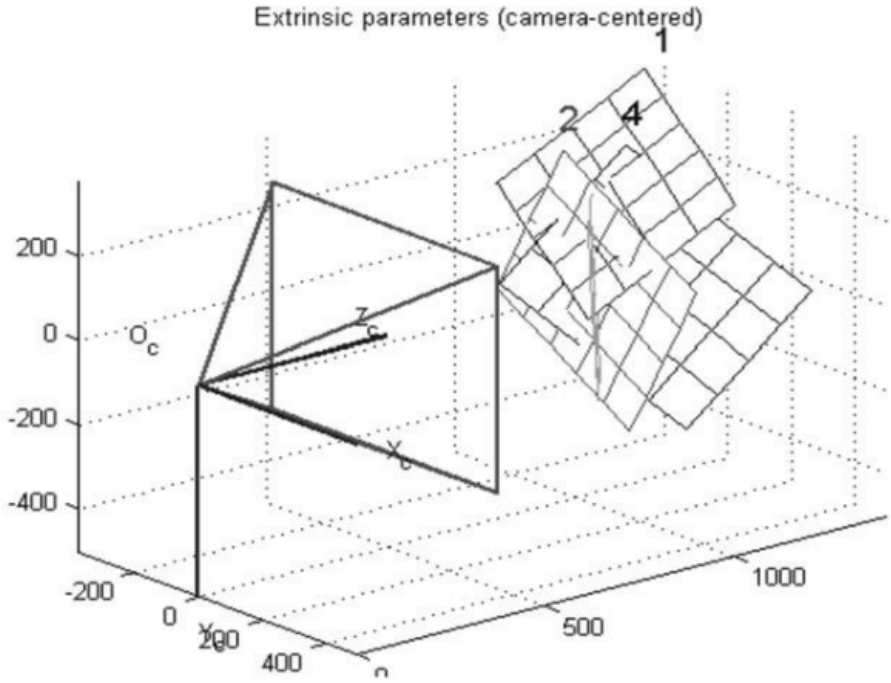
UNIVERSITÀ DI PARMA

- Not very practical to use 3D calibration rigs…
- Usually calibration grids are “flat” (Matlab, OpenCV)
- This does not fit well with Tsai’s requirements
  - No coplanar points
- Zhang’s approach, conversely, exploits this!

Zhang approach (2000)

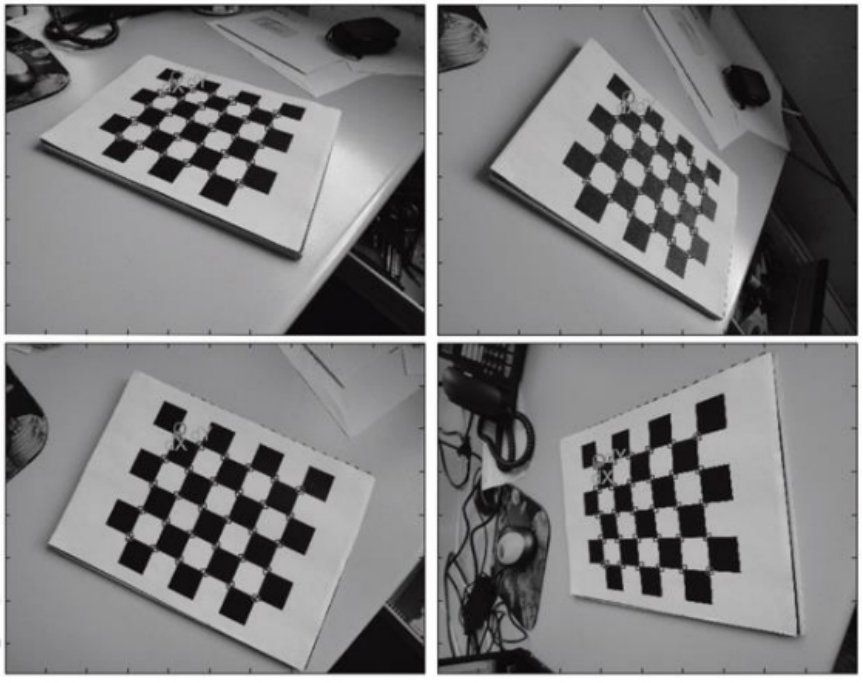


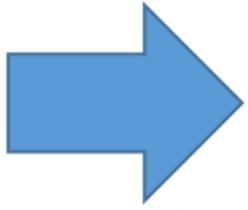


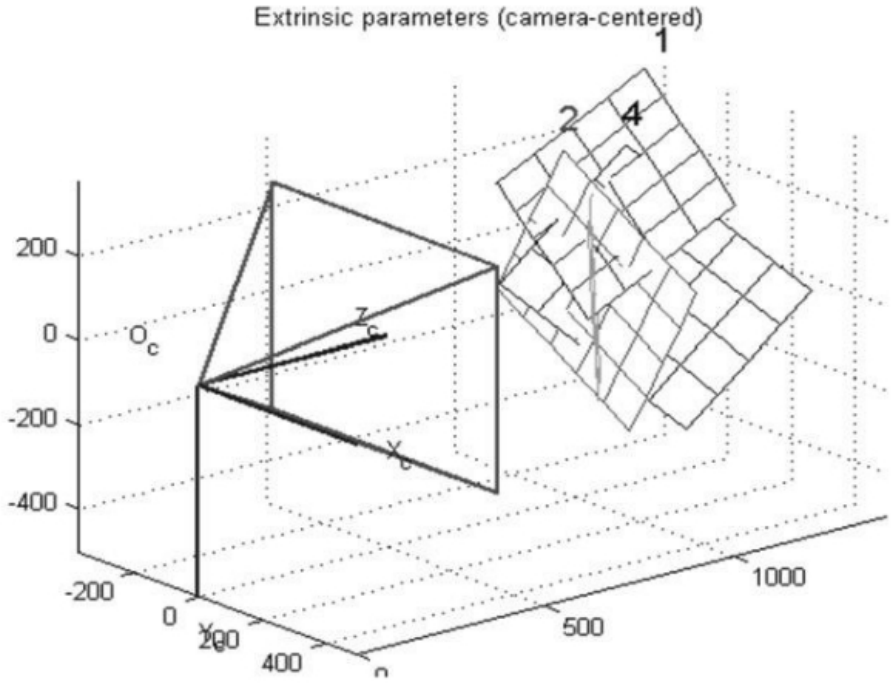


Zhang, *A flexible new technique for camera calibration*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000.

Zhang approach (2000)

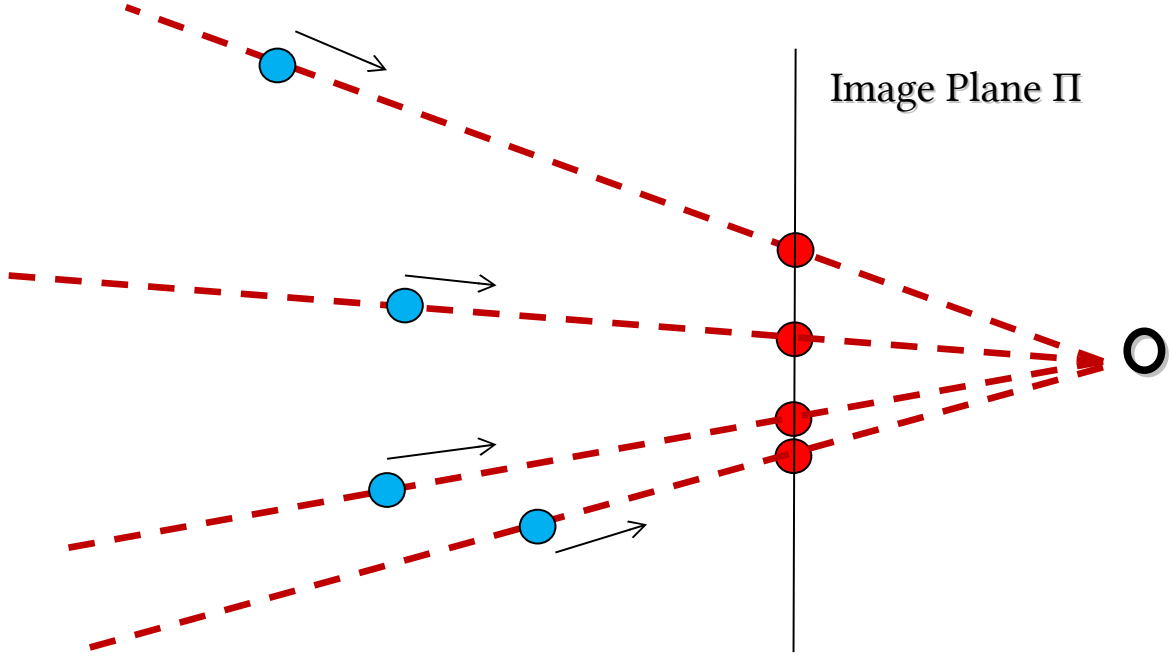






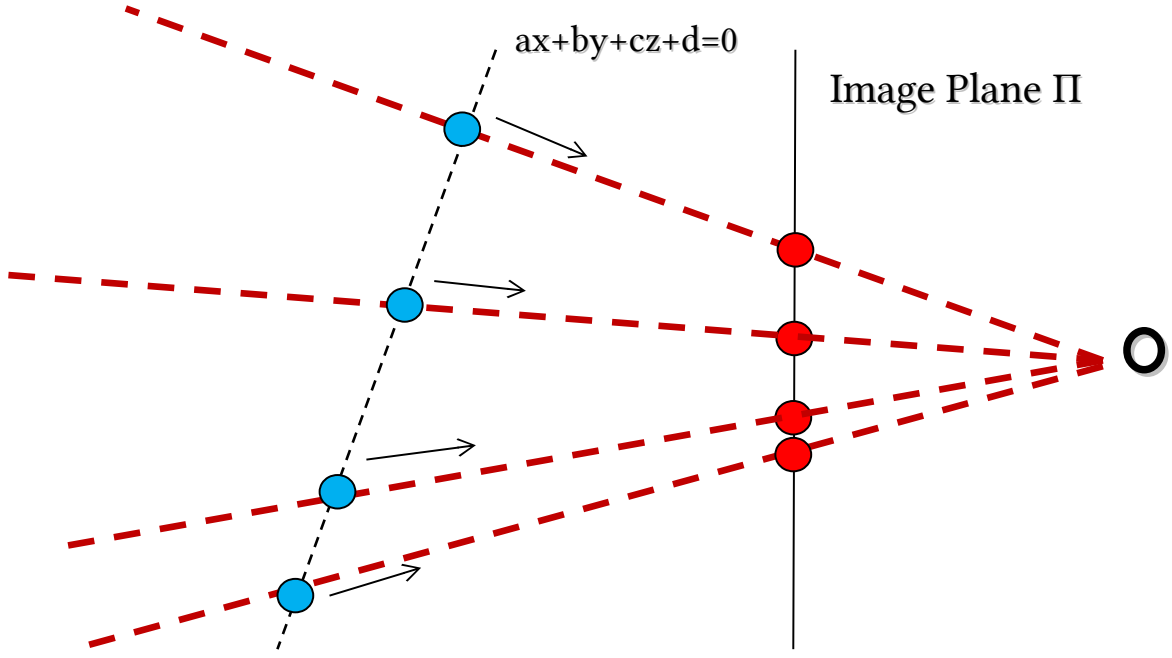
Before detailing Zhang's approach, just understand what does it mean to frame a "plane"

Ill posed problem



- Projective space ⟶ we lost one dimension
- 3D [x,y,z] are projected in [x/z,y/z,1]

Add a constraint



- Just image that the (red) image points are projection of points that lie on the same plane
- In such a case for each image point there is only one world point (cyan)

Add a constraint

UNIVERSITÀ DI PARMA

- Given a plane aX + bY + cZ + d = 0

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} \overset{3\times4}{=} M \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad \rightarrow \quad \begin{bmatrix} u \\ v \\ w \\ 0 \end{bmatrix} = \overset{4\times4}{\begin{bmatrix} M \\ a\,b\,c\,d \end{bmatrix}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = \begin{bmatrix} M \\ a\,b\,c\,d \end{bmatrix}^{-1} \begin{bmatrix} u \\ v \\ 1 \\ 0 \end{bmatrix} \quad \overset{euclidean}{\rightarrow} \quad \begin{bmatrix} X/W \\ Y/W \\ Z/W \end{bmatrix}$$

- We can obtain euclidean world coordinates of pixel (u,v)

Specific case, plane Z=0

UNIVERSITÀ DI PARMA

- Consider a very specific plane ⟶ Z=0

$$\begin{bmatrix} u \\ v \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = H \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

- H is 3×3
- H is an **Homography**
- This actually works for every possible plane!

Better understanding H

UNIVERSITÀ DI PARMA

- We started from a general M but
  - M = K [R T]

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} \alpha & -\alpha\cot\theta & c_x \\ 0 & \dfrac{\beta}{\sin\theta} & c_y \\ 0 & 0 & 1 \end{bmatrix}}^{K_{3\times 3}} \overbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}}^{[RT]_{3\times 4}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Better understanding H

UNIVERSITÀ DI PARMA

- When the Z = 0 we can simplify as:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ r_{31} & r_{32} & t_z \end{bmatrix}_{3\times 3} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Better understanding H

UNIVERSITÀ DI PARMA

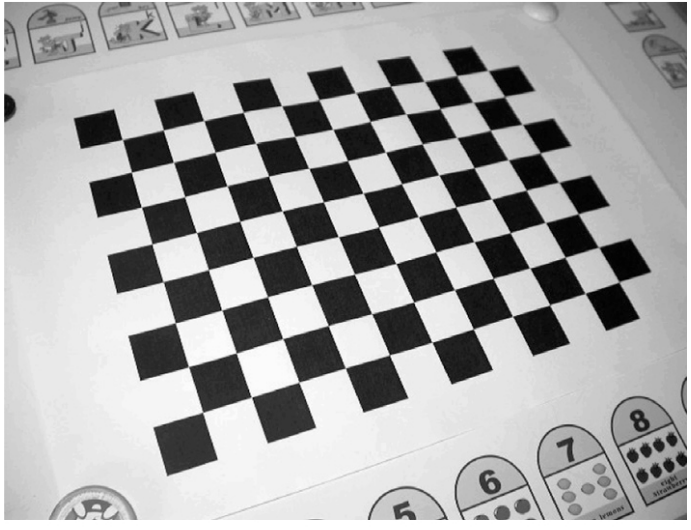
- When the Z = 0 we can simplify as:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ r_{31} & r_{32} & t_z \end{bmatrix}_{3\times 3} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \overbrace{K \begin{bmatrix} r_1 r_2 t \end{bmatrix}}^{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Zhang's approach

UNIVERSITÀ DI PARMA

- Zhang idea is to use a "planar" calibration checkboard
  - Set the world coordinate system onto the checkboard ⟶ z = 0 plane…
- In such a case we can apply an homography!



Calibration

UNIVERSITÀ DI PARMA

- For each point on the planar surface of my grid I then obtain
- In the same image…

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = H \begin{bmatrix} X_i \\ Y_i \\ 1 \end{bmatrix} \quad \text{with} \quad i = 1, 2, 3, \ldots, n$$

Calibration

UNIVERSITÀ DI PARMA

- Again a system of n equations
  - Unknowns are H elements

$$QH = 0$$

Calibration

UNIVERSITÀ DI PARMA

- For solving this issue we can use the same approach used for “Tsai”
- We have a 3×3 homograpy instead of a 3×4 projection matrix
- We need to solve $QH = 0$ where Q is $2n \times 9$ and has rank 8
- We need at least 4 points
  - When $n > 4 \longrightarrow$ SVD

Calibration

UNIVERSITÀ DI PARMA

- For computing H we can, again, use a SVD
- It is, again, an homogeneous system

$$QH = 0$$

- Is H really useful?
  - No, given they mix intrinsic parameters and very specific extrinsic ones
  - I put my coords system on the grid!

Computing H

UNIVERSITÀ DI PARMA

- Each view has different H but **K is the same for all views**

$$H^i = K\left[r_1^i r_2^i t\right]$$

- Once computed H^i we can focus on $H^i \leftrightarrow K$ relation

Computing K given H

UNIVERSITÀ DI PARMA

- K can be anyway computed from H
  - Constraints: r_1, r_2 form an orthonormal basis
    - This allows to compute$K^{-T}K^{-1}$
  - Cholesky decomposition can be applied to the symmetric and positive $B=K^{-T}K^{-1}$ for computing K^T
  - Constraints: we need multiple views
    - We will discuss this later (without details)

Computing extrinsic parameters

UNIVERSITÀ DI PARMA

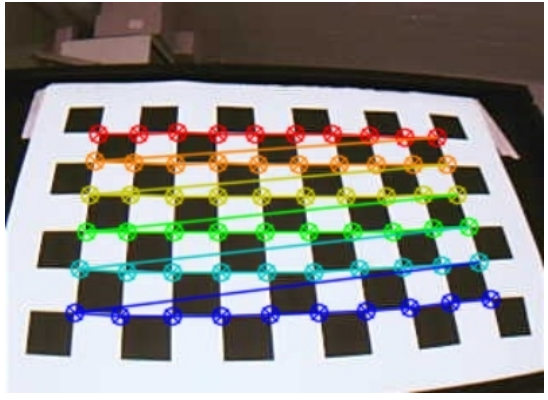
- We now have multiple H and one K
- We can compute extrinsic parameters of each view from them…
  - A scale factor is, again, to be considered…
  - λ

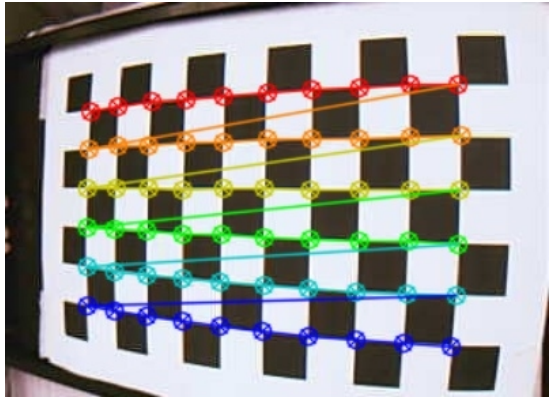
$$
\begin{aligned}
r_1 &= \lambda K^{-1} h_1 \\
r_2 &= \lambda K^{-1} h_2 \\
r_3 &= r_1 \times r_2 \\
t &= \lambda K^{-1} h_3
\end{aligned}
$$

How many grids?

UNIVERSITÀ DI PARMA

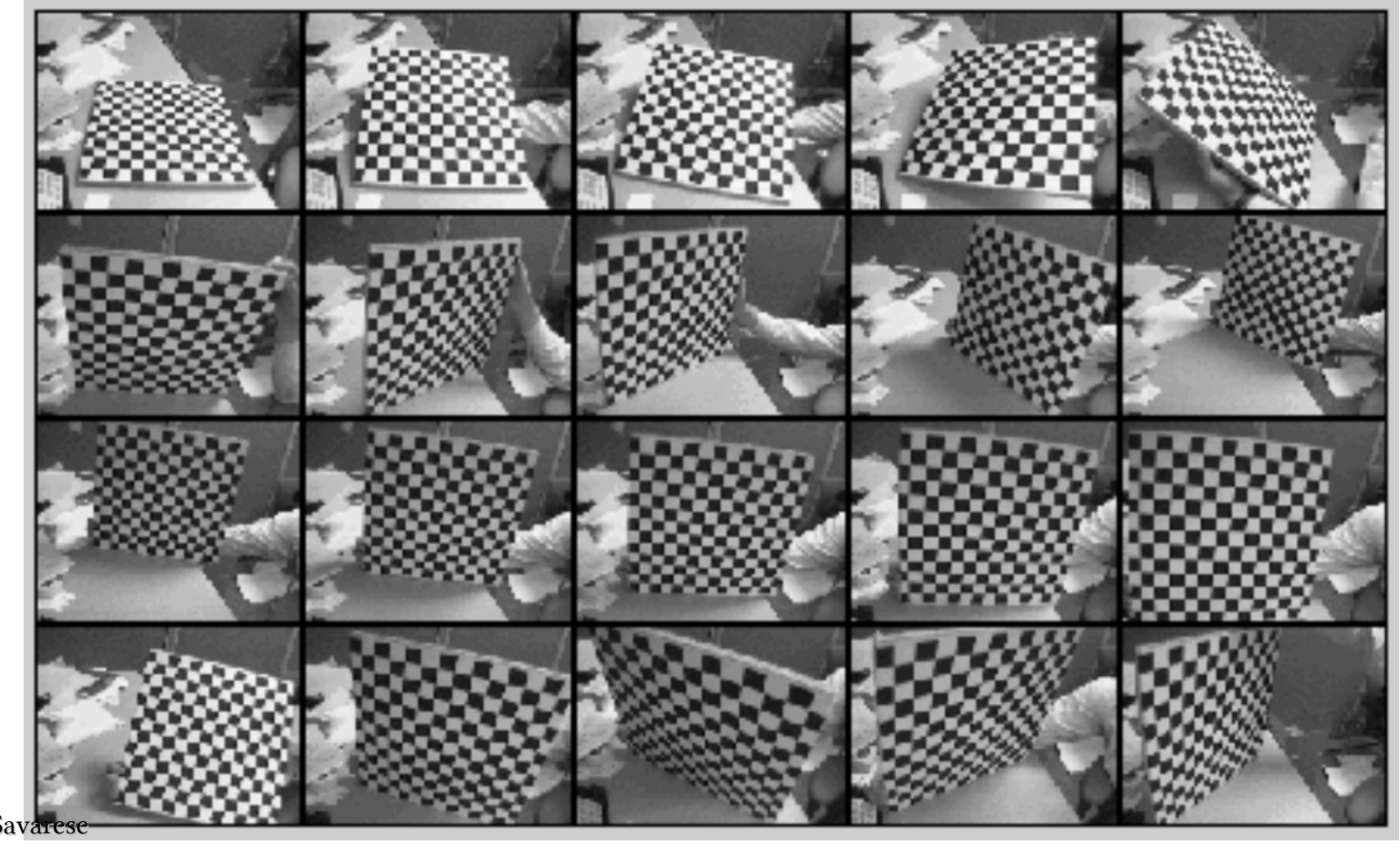
- With a “single” H we can not anyway fully compute K
- 3 Hs are needed, at least, to compute K
  - The larger the number of views, the better the result
  - Usually 20-50 grid views





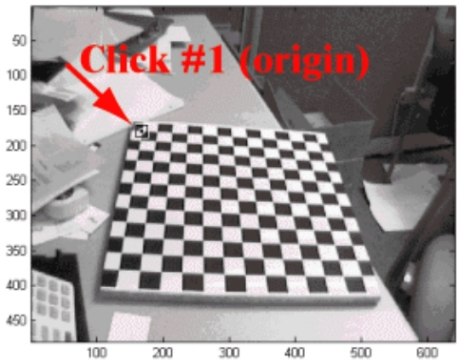
Example of calibration data

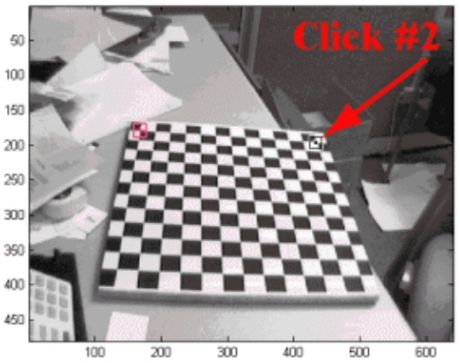
UNIVERSITÀ DI PARMA

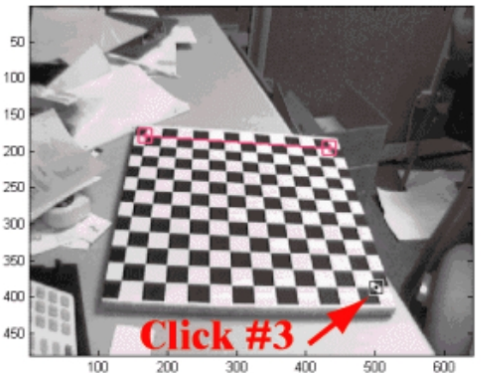


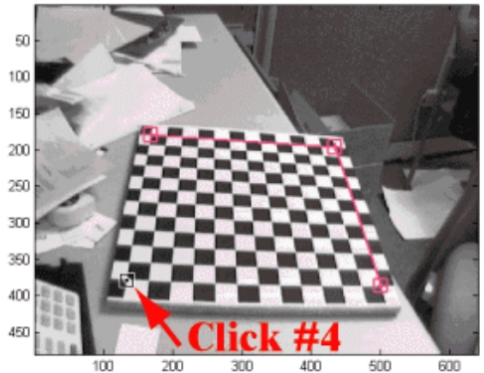
Courtesy: Silvio Savarese

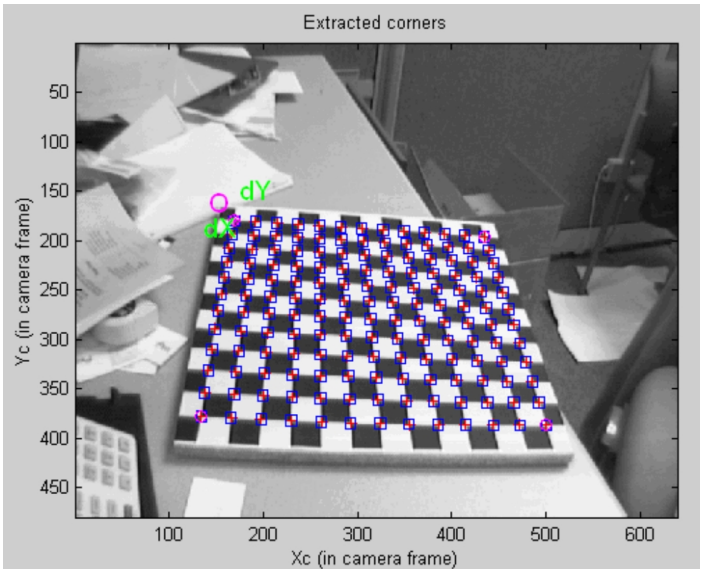
Example of calibration data









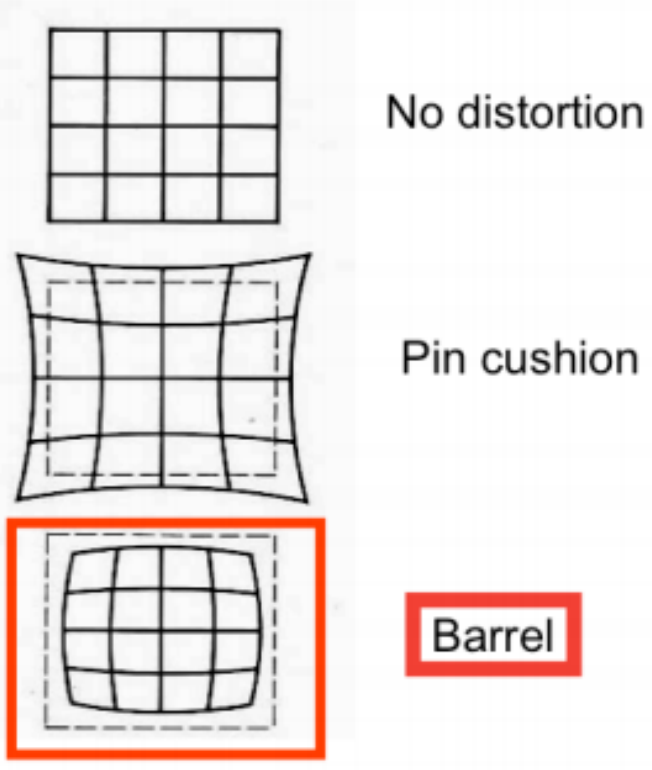


Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

Lens Distortion

Lens Distortion

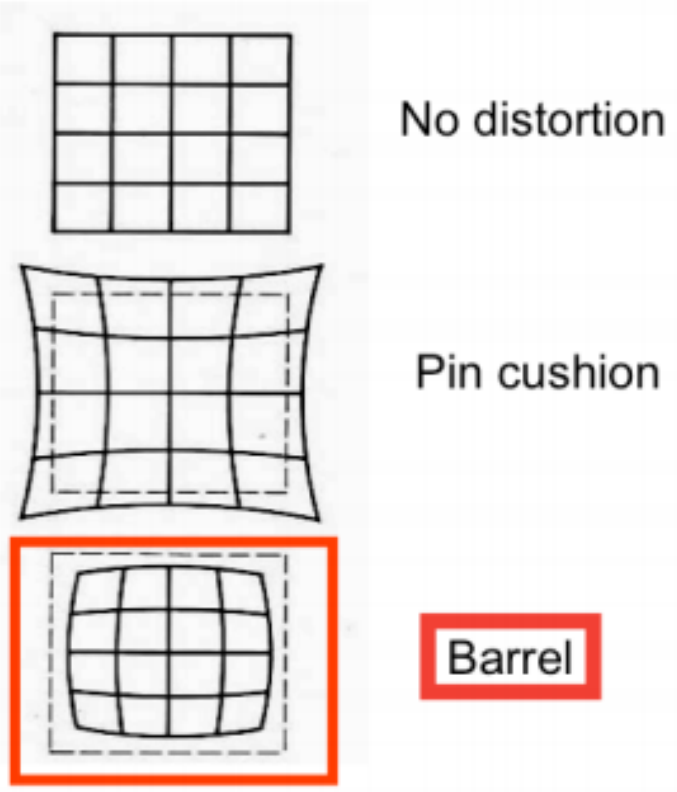


Courtesy: Silvio Savarese

Radial Lens Distortion

UNIVERSITÀ DI PARMA

- We assumed a “thin” lens
  - This basically means that lines are lines
  - Unfortunately this is not true!
- Radial Distortion
  - Image magnification/contraction depends on distance from optical axis
  - Deviations are more evident on lateral areas
  - Cheap optics

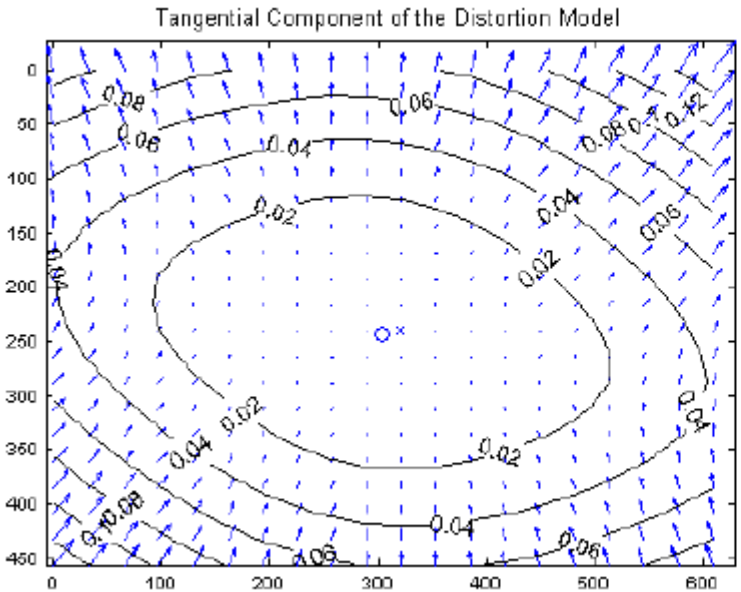


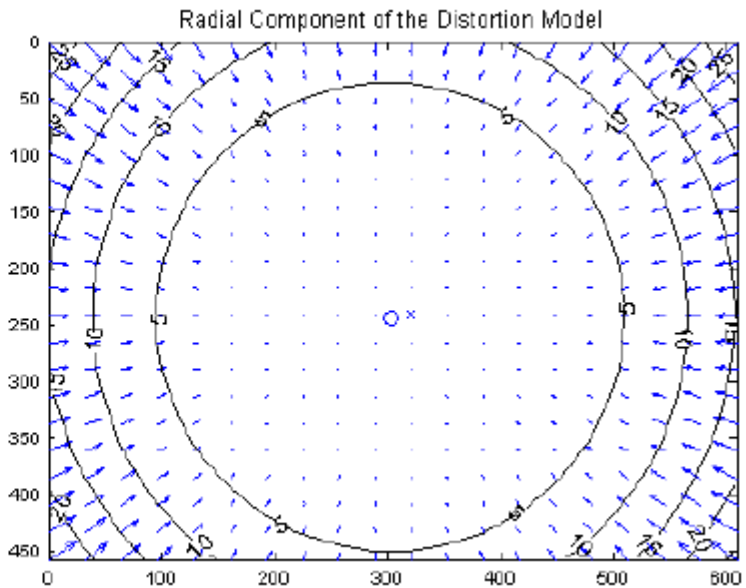
Courtesy: Silvio Savarese

Tangental Lens Distortion

UNIVERSITÀ DI PARMA

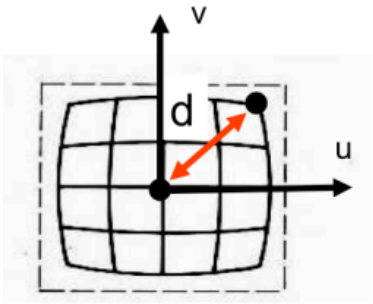
- Typical of misaligned lenses
  - Not parallel to image sensor





Courtesy: Technische Universität München

Radial Lens Distortion



$$S_\lambda = \begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad M\,P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i$$

Distortion coefficient

$$\lambda = 1 \pm \sum_{p=1}^{3} \kappa_p d^{2p} \qquad d^2 = a\,u^2 + b\,v^2 + c\,u\,v$$

[Eq. 5] Polynomial function — To model radial behavior [Eq. 6]

- λ is the distortion factor
  - Actually it is a $\lambda_{(u,v)}$ since it depends on the distance from optical axis

Courtesy: Silvio Savarese

Radial Lens Distortion

$$\underbrace{\begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{Q} M \quad P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i \qquad Q = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_3 \end{bmatrix}$$

- λ depends on u,v and this leads to a non linearity in the $P \rightarrow p$ mapping

Courtesy: Silvio Savarese

Radial Lens Distortion

$$\underbrace{\begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{Q} M P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i \qquad Q = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_3 \end{bmatrix}$$

- Tsai 87, initially estimate m_1 and m_2 by SVD and then $m_3=f(m_1, m_2, \lambda)$

Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

OpenCV Calibration

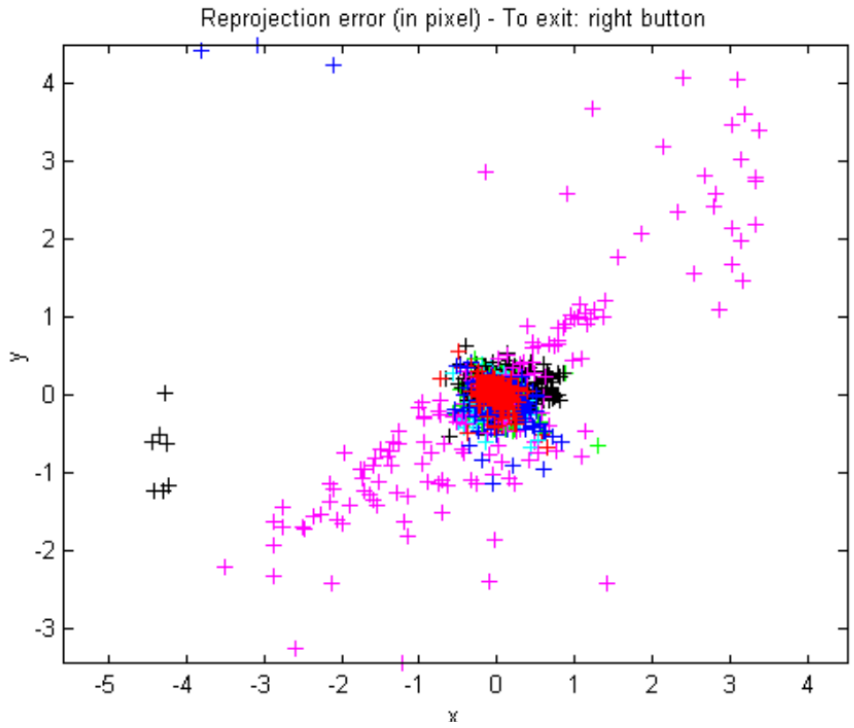
Multi plane calibration

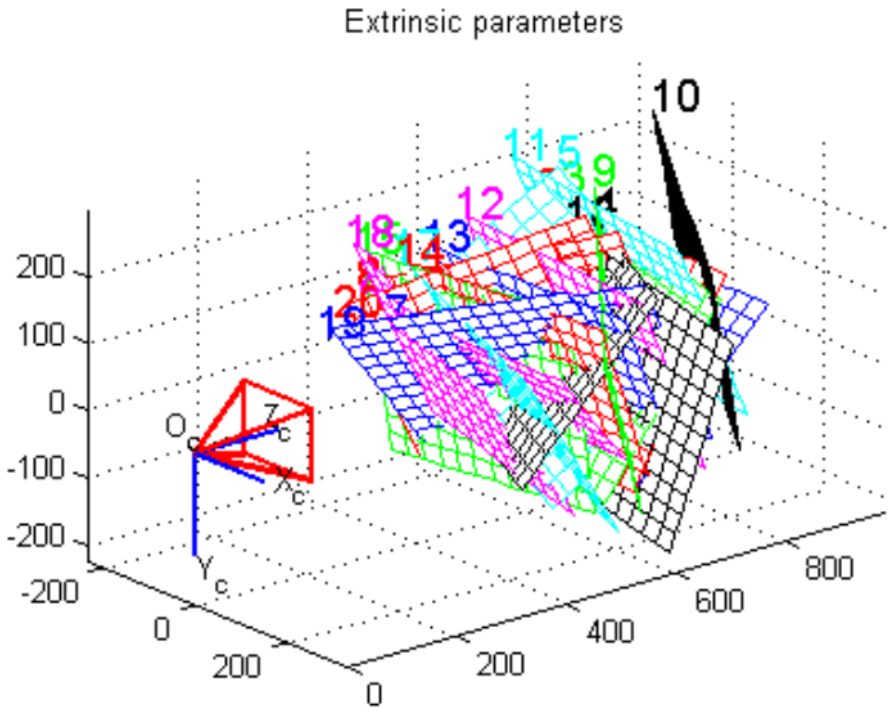
UNIVERSITÀ DI PARMA

- Requires a plane only!
- No known positions/orientations
- Strobl et al. approach (2011)

- Free code:
  - https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

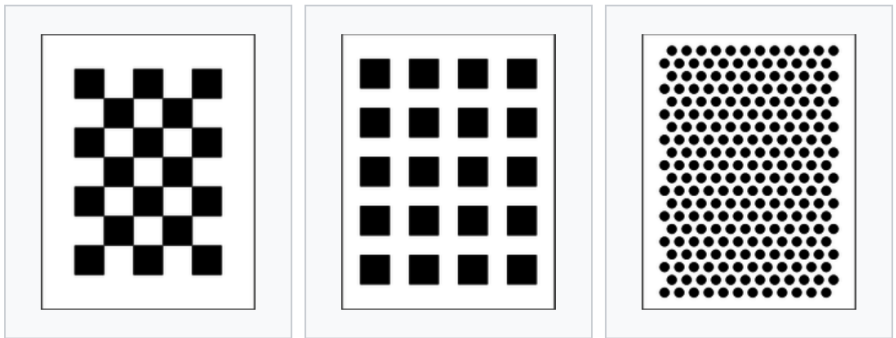
Example of calibration data

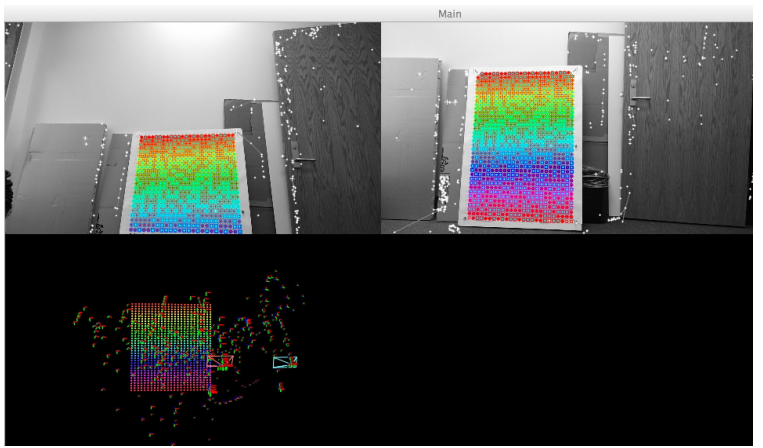




Courtesy: Silvio Savarese

Calibration Patterns





https://github.com/arpg/Documentation/tree/master/Calibration

https://boofcv.org/index.php?title=Tutorial_Camera_Calibration

UNIVERSITÀ DI PARMA

Perspective-n-Point

Extrinsic parameters

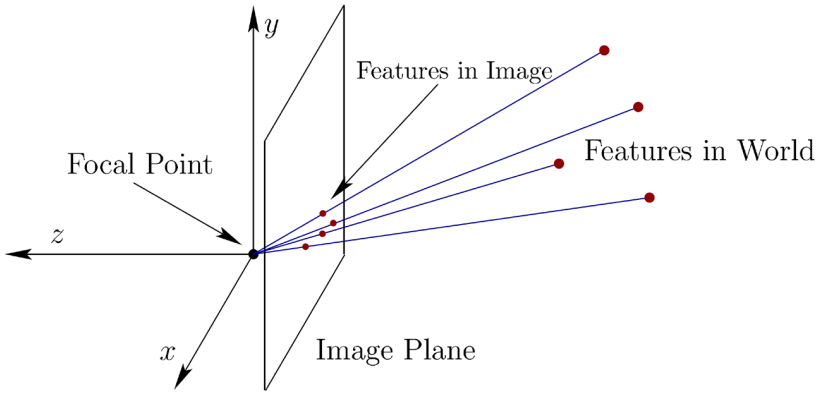
UNIVERSITÀ DI PARMA

- Tsaj, Zhang, and other methods allow to easily compute K
  - Intrinsic parameters “only”
- General problem:
  - I already computed K
  - I want to compute or update [RT]
  - 6 DOF (3 translation, 3 rotation)
- Namely, camera pose estimation “only”

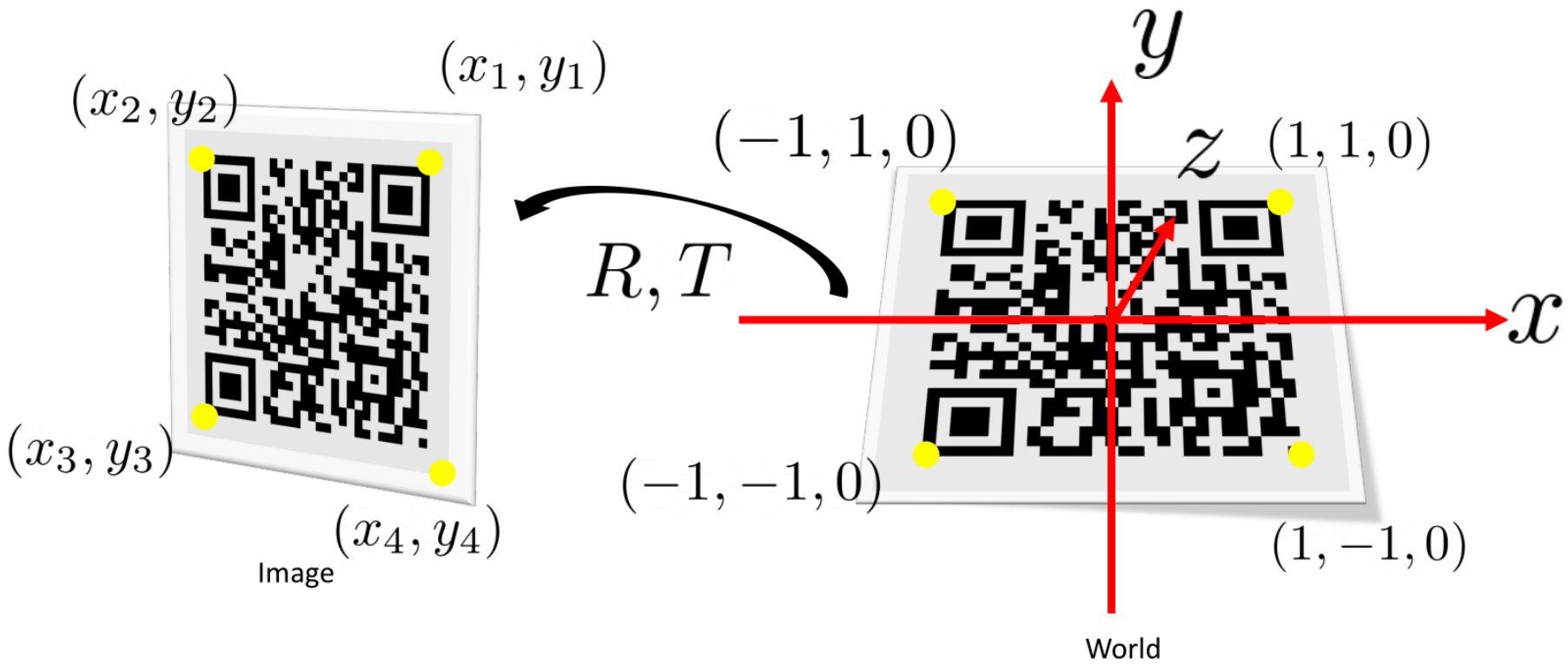
Perspective-n-Point

UNIVERSITÀ DI PARMA

- We know:
  - A set of *n* world coordinates P_w
  - Their projections p_c on an image
  - Camera Intrinsics K
- Unknowns
  - 3D camera rotation wrt world coordinates R
  - 3D camera translation wrt wolrd reference origin
- Different approaches



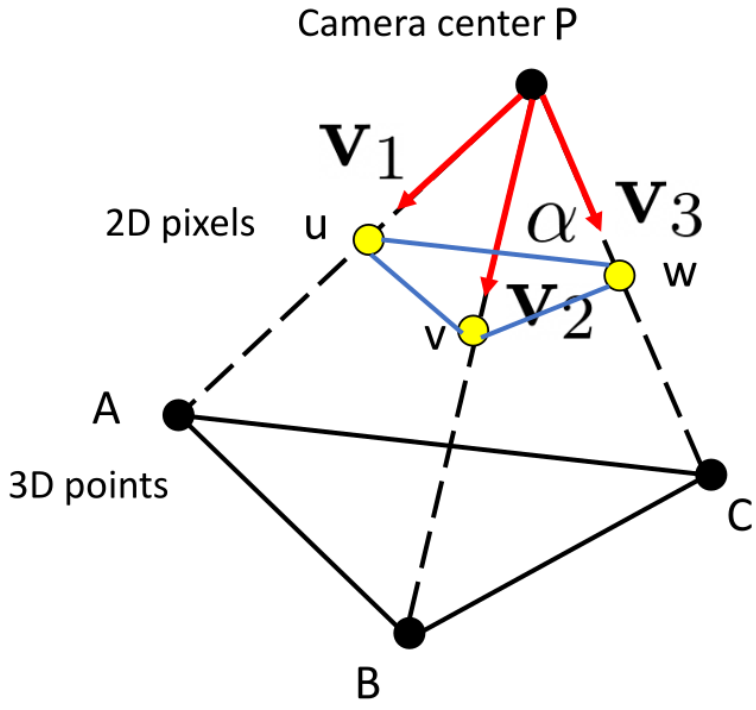
PnP Problem with QR code



Courtesy: Yu Xiang

- I can use only 3 known points (A, B, C)
- And their projections u, v, w

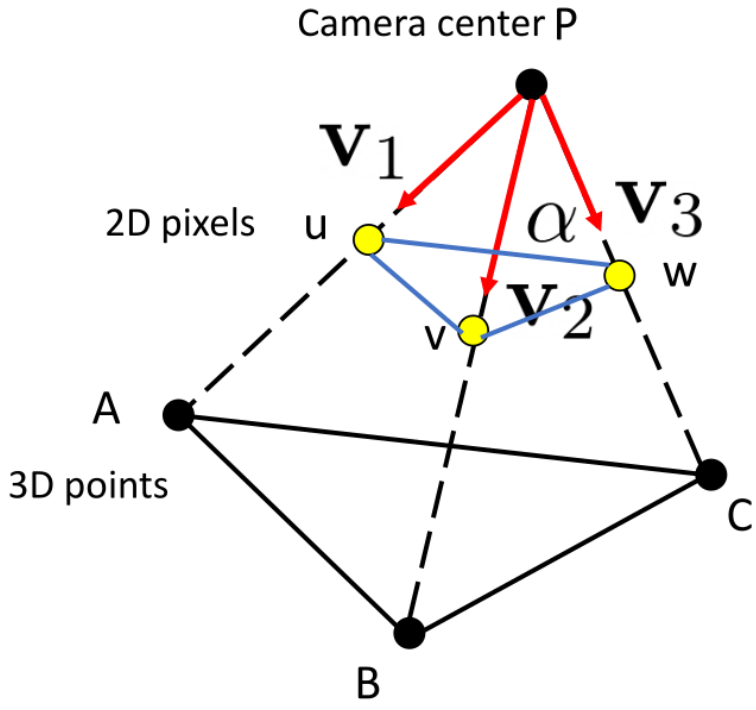
$$
\begin{aligned}
u &= K\left[I\,0\right]A \\
v &= K\left[I\,0\right]B \\
w &= K\left[I\,0\right]C
\end{aligned}
$$



Courtesy: Yu Xiang

- K^{-1} can be computed for sure
- If I multiply K^{-1} for u, v, w what is the result?
  - Lines of sight

$$\vec{v}_1 = K^{-1}u$$
$$\vec{v}_2 = K^{-1}v$$
$$\vec{v}_3 = K^{-1}w$$

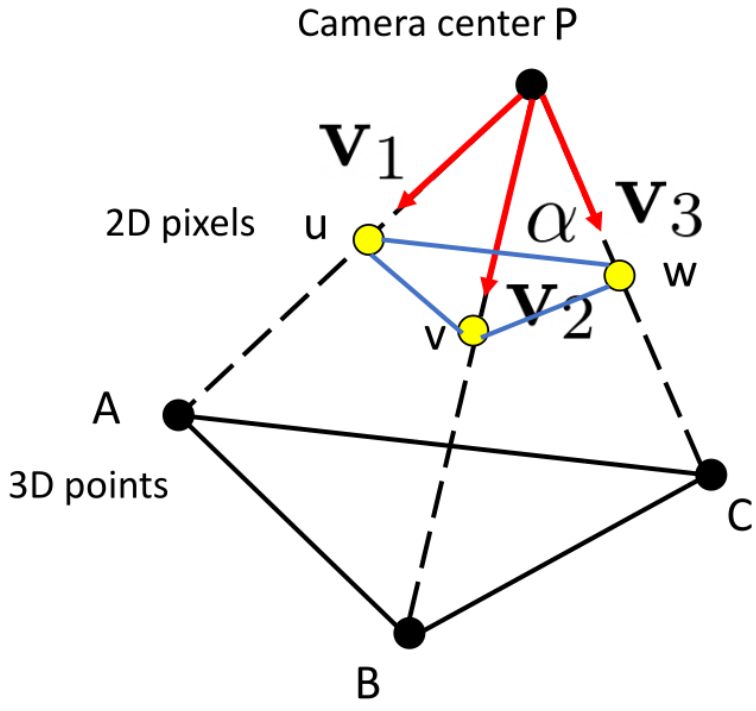


Courtesy: Yu Xiang

- Using the 3 vectors we can also compute the angles between them
- As example:

$$\cos(\alpha) = \frac{\vec{v}_2 \cdot \vec{v}_3}{\|v_2\| \ \|v_3\|}$$

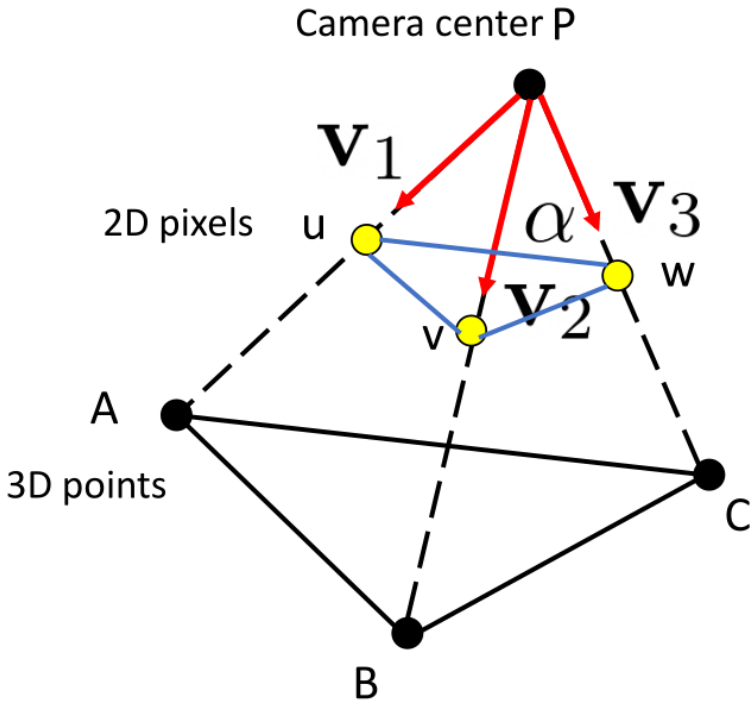
- The same for other angles (γ and β)



Courtesy: Yu Xiang

- We can compute the distance of points A, B, and C from P exploiting:

$$\overline{AB}^2 = \overline{PA}^2 + \overline{PB}^2 - 2\cos(\gamma)$$
$$\overline{AC}^2 = \overline{PA}^2 + \overline{PC}^2 - 2\cos(\beta)$$
$$\overline{BC}^2 = \overline{PB}^2 + \overline{PC}^2 - 2\cos(\alpha)$$



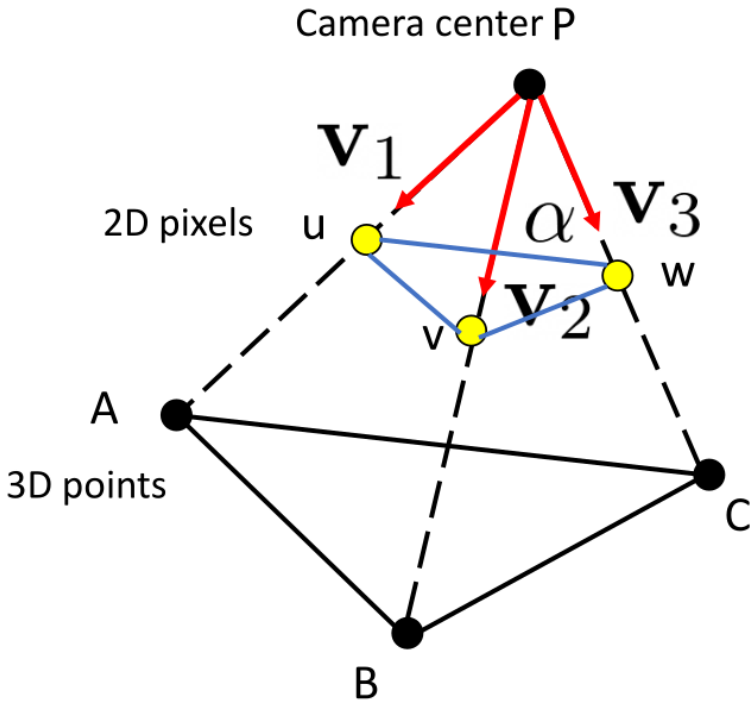
- PA, PB, and PC are the unknowns!

Courtesy: Yu Xiang

P3P

UNIVERSITÀ DI PARMA

- We can solve previous equation system but we obtain 4 possible solutions…
- A 4th point can be used which one aligns most accurately
  - We should rather say P3+1P
  - No exact solution for sure (noise)
- Given the relative position between P and the 3 world points we can recover camera pose



Courtesy: Yu Xiang

P3P with RANSAC

UNIVERSITÀ DI PARMA

- RANSAC is a method to estimate parameters from noisy observed data
- Yes, our measurement are for sure noisy!
- Instead of using 3+1 points, use N points where N >> 3
  - Randomly select 3 points
  - Use P3P to compute camera pose ⟶ current hypothesis
  - Count how many points support current hypothesis
  - Go back to point selection until we found a suitable solution or sufficient trials

EPnP

- An accurate solution to the PnP problem
  - E $\longrightarrow$ efficient
  - Non iterative
  - O(n) wrt O(n^x) with $5 \leq x \leq 8$
  - Slightly less precise than iterative methods
  - ICCV 2007

- cv::solvePnP(), different methods:
  - cv::SOLVEPNP_P3P or cv::SOLVEPNP_AP3P: 4 input points
  - cv::SOLVEPNP_IPPE: >= 4 coplanar points
  - cv::SOLVEPNP_IPPE_SQUARE: 4 coplanar points on thje corners of a square (QR code)
- cv::solvePnPRansac(): more than 4 points
- Other refinement methods

UNIVERSITÀ DI PARMA

Camera Models (2)

Question time!

UNIVERSITÀ DI PARMA

Camera Models (2)

CAMERA MODELS (2)

Summary

UNIVERSITÀ DI PARMA

- Pin Hole Camera recap
- Calibration

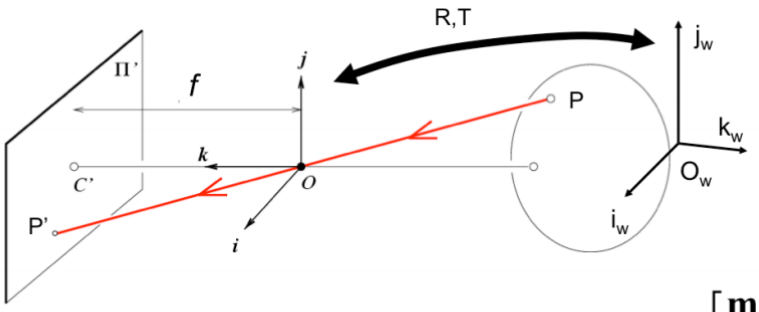
UNIVERSITÀ DI PARMA

Pin Hole Model recap

Piccolo recap e integrazione precedente lezione

Perspective Transformation

UNIVERSITÀ DI PARMA



$$P'_{3\times1} = M\,P_w = K_{3\times3}\begin{bmatrix} R & T \end{bmatrix}_{3\times4} P_{w\,4\times1} \qquad M = \begin{bmatrix} \mathbf{m}_1 \\ \mathbf{m}_2 \\ \mathbf{m}_3 \end{bmatrix}$$

$$= \begin{bmatrix} \mathbf{m}_1 \\ \mathbf{m}_2 \\ \mathbf{m}_3 \end{bmatrix} P_W = \begin{bmatrix} \mathbf{m}_1 P_W \\ \mathbf{m}_2 P_W \\ \mathbf{m}_3 P_W \end{bmatrix} \quad \mathbf{E} \rightarrow \left(\frac{\mathbf{m}_1 P_w}{\mathbf{m}_3 P_w}, \frac{\mathbf{m}_2 P_w}{\mathbf{m}_3 P_w}\right) \text{ [Eq.12]}$$

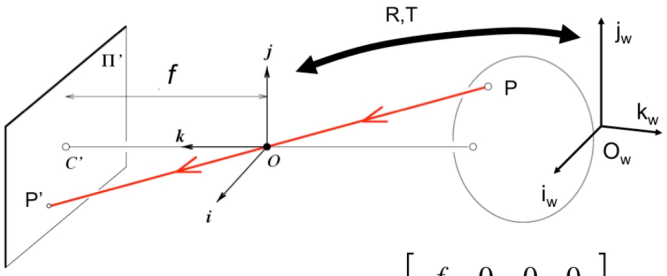
Courtesy: Silvio Savarese

This slide summarizes the main equation that describes the projective transformation from point Pw in the world coordinate system (in homogenous coordinates) into a point PE in the image pixels (in Euclidean coordinate systems).
Eravamo arrivati qui. Il legame tra punto mondo e punto immagine espresso come moltiplicazione di una matrice con il vettore delle coordinate mondo (omogenee)
E come ritornare in quelle cartesiane…

Perspective Transformation (ideal case)

UNIVERSITÀ DI PARMA

- Simplified situation: no rotation, no translation, no skew, no offset, squared pixels



$$M = K\begin{bmatrix} R & T \end{bmatrix} = K\begin{bmatrix} I & 0 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\rightarrow P'_E = \left(\frac{\mathbf{m}_1 P_w}{\mathbf{m}_3 P_w}, \frac{\mathbf{m}_2 P_w}{\mathbf{m}_3 P_w}\right) = \left(f\frac{x_w}{z_w}, f\frac{y_w}{z_w}\right) \qquad P_w = \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Courtesy: Silvio Savarese

Let's consider now the following exercise: suppose that the two reference systems coincide (R=I3x3 and T=03x1) and the camera model has focal length f, zero-skew, no offset and square pixels. Can we write a simplified expression for P'E? Notice that the result is exactly what we derived for the pinhole camera in lecture 2.

In pratica mi mostra il modello semplificato in forma di prodotto matriciale in coordinate omogenee.

Questa non l'avevamo vista ma accennata, È un caso particolare di camera ideale

Perspective Transformation

UNIVERSITÀ DI PARMA

- 11 degrees of freedom

$$P' = M P_w = K[R \quad T] P_w$$

Internal parameters

External parameters

$$\mathcal{M} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix}_{3\times 4}$$

$$K = \begin{bmatrix} \alpha & -\alpha\cot\theta & u_o \\ 0 & \frac{\beta}{\sin\theta} & v_o \\ 0 & 0 & 1 \end{bmatrix} \quad R = \begin{bmatrix} \mathbf{r}_1^T \\ \mathbf{r}_2^T \\ \mathbf{r}_3^T \end{bmatrix} \quad T = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

Courtesy: Silvio Savarese

Tecnicamente posso misurare (quasi) tutto, inclinometro, metro ecc. ecc.
Ma commetto molti errori. Molto meglio osservare come la camera proietta i punti mondo nell'immagine e dedurre i parametri → **calibrazione**

I parametri da ricavare sono 11‼

Recall that our camera is modeled using internal, or intrinsic, parameters (K) and external, or extrinsic, parameters ([R T]). Their product gives us the full 3x4 projection matrix M, shown above in the expanded form. It has 11 degrees of freedom (we discussed that in lecture 2). R is expressed as function of r1T, r2T and r3T which are row vectors. T has coordinates t
x, ty, tz

UNIVERSITÀ DI PARMA

Calibration
Tsai approach (1987)

Per calibrare si possono usare due approcci differenti, vediamoli brevemente

Calibration

UNIVERSITÀ DI PARMA

$$P' = M P_w = K [R \quad T] P_w$$

Internal parameters

External parameters

- Calibration → **estimation of intrinsic/extrinsic parameters**
- We need 1 or, rather, more images

Change notation:
$P = P_w$

$p = P'$

Adesso inizieremo a vedere un possibile modo di calibrazione. Ocio che di fatto non è utilizzato nella pratica, c'è di meglio e chi segue il corso ADAS lo vedrà….

Prima di tutto però un cambio notazione. Da adesso in poi il punto mondo diventa P grande e il punto immagine p piccolo (ok è seccante, ma c'è di peggio nella vita eh?)

We now pose the following problem: given one or more images taken by a camera, estimate its intrinsic and extrinsic parameters. Notce that from this point on we change the notaton for Pw which now is denoted as P, and P' which is now denoted as p.

Calibration

UNIVERSITÀ DI PARMA

- Small notation change
  - $P_W \longrightarrow P$
  - $P' \longrightarrow p$

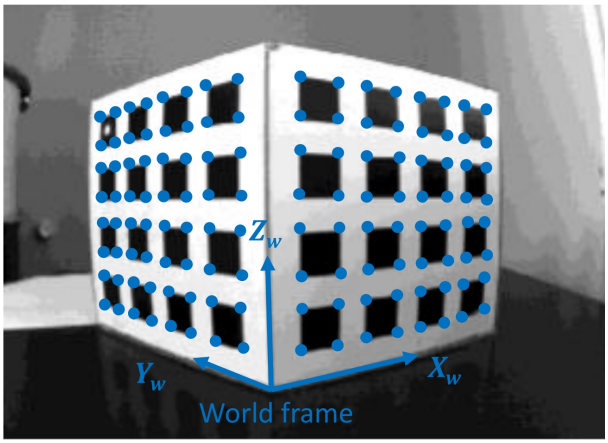
$$p = K\left[RT\right]P = MP$$

Quindi dato il cambio di notazione cambia la mia formola

Tsai's method

UNIVERSITÀ DI PARMA

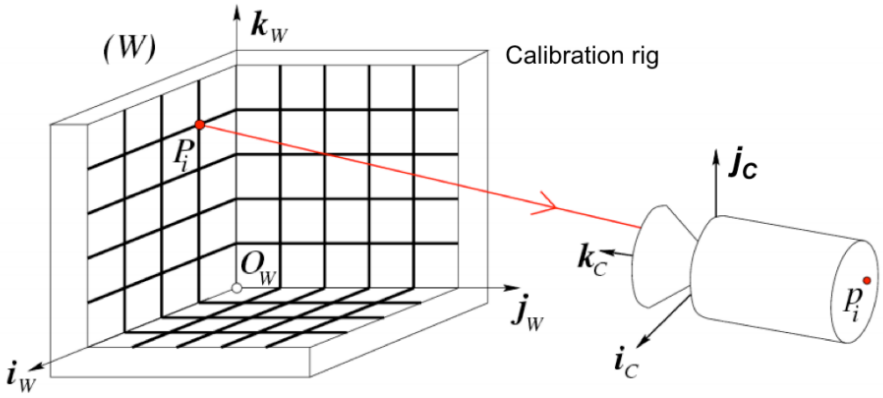
- We use 3D calibration targets and their projection on an image



Il metodo di Tsai si basa su pattern "ortogonali"

Calibration

UNIVERSITÀ DI PARMA



- We need: n points (P_1, P_2,…,P_n) in known positions [O_w,i_w,j_w,k_w] in the world reference system

Courtesy: Silvio Savarese

Detto in altri termini: posiziono dei punti in differenti posizioni del mondo in posizioni note. Chiaramente devono essere riconoscibili nell'immagine e anche distinguibili tra di loro.
Anzi è opportuno siano altamente riconoscibili nella immagine.
Per questo motivo uso la cosiddetta GRIGLIA DI CALIBRAZIONE (in questo caso griglie ortogonali tra di loro)

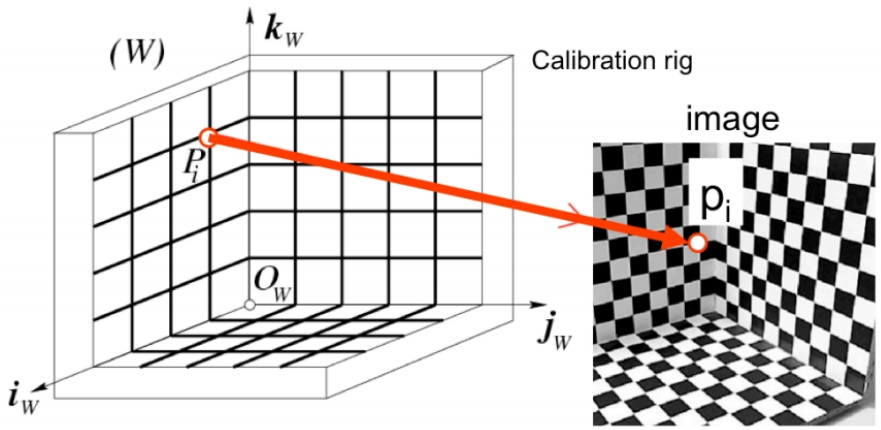
La griglia è in posizione nota e spesso uso griglie con 3 piani tra di loro ortogonali anzi la centro sull'origine e faccio in modo che I piani corrispondano a quegli degli assi del mio sistema di riferimento mondo (già questo ci fa capire il perché non sia usatissimo come metodo per calcolare gli estrinseci visto che questo allineamento non è banale, ma ci semplifica I restanti calcoli)

Seleziono un certo numero (sì ma quanti?) punti nel mondo ovvero punti sulla griglia

We can describe this problem more precisely using a calibraton rig, such as the one shown above. The rig usually consists of a paeern (usually a checkerboard) with known dimensions.
The rig also defines our world reference coordinate frame ([Ow,iw,jw,kw] as shown in the figure.

Calibration

UNIVERSITÀ DI PARMA



- We also need: camera coordinates for the n points projections (p_1, p_2,…,p_n) as (i_c,j_c)

Courtesy: Silvio Savarese

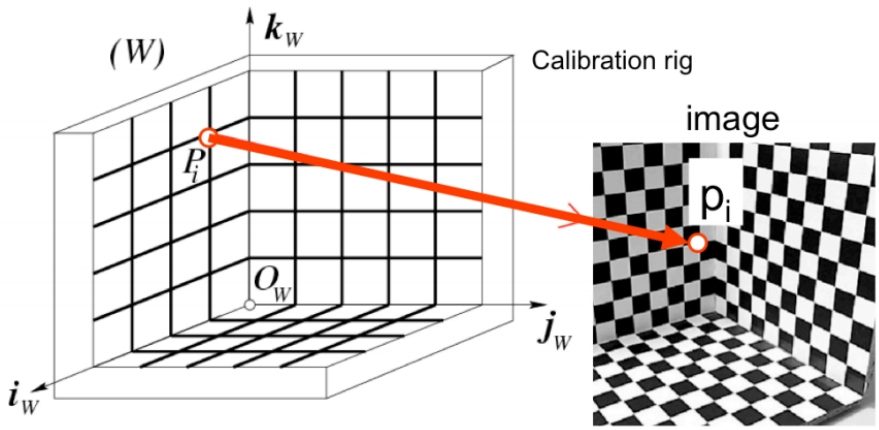
Come vedete nell'immagine una possibile griglia è costituita da quadrotti bianchi e neri che mi permettono di individuare in maniera relativamente semplice gli "incroci"

Dati gli n punti selezionati nel mondo io ricavo I corrispondenti n punti nell'immagine

Now, suppose we are given n points on the calibration rig, P1… Pn, along with their corresponding coordinates, p1, … pn in the image. Using these correspondences, our goal is to estmate both intrinsic and extrinsic parameters. Notice that P1… Pnas well as p1, … pn are known!

Calibration

UNIVERSITÀ DI PARMA



- 11 degrees of freedom → how many matches we need?
- Each correspondence → 2 equations (world coordinates → row/column)
- We need, at least, 6 correspondences

Courtesy: Silvio Savarese

Ma n cosa vale? Ovvero quanto punti devo selezionare?

Devo ricavare 11 parametri. Per ogni punto mondo e punto immagine io ho $p_i=MP_i$, che mi porta a 2 equazioni che legano una x immagine a x,y,z mondo e y immagine a x,y,z mondo ovvero due equazioni.

Con 6 corrispondenze ricavo 12 equazioni e quindi dovrei riuscire a ricavare 11 parametri. Anzi ce ne è pure una di troppo.

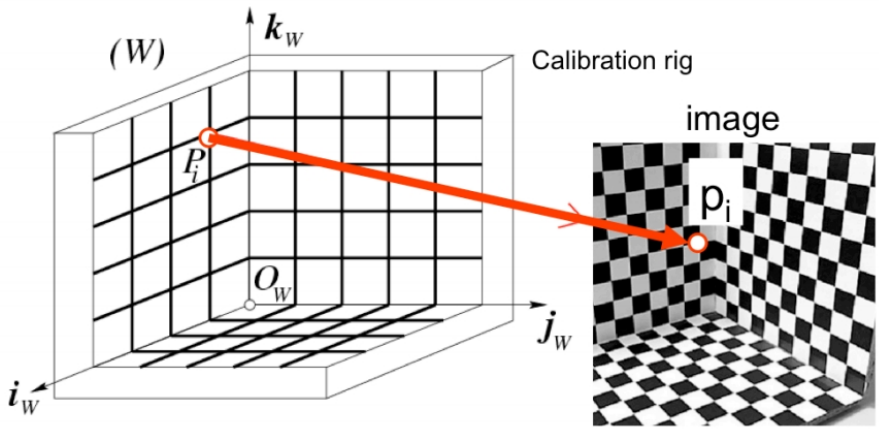
Sì, in teoria sì. All'atto pratico no. E perché no? Ma perché I pixel sono quantizzati e quindi la loro posizione immagine è giocoforza imprecisa. Inoltre sarò stato preciso fin che voglio a misurare le coordinate mondo ma sicuramente avrò fatto errori ecc. Ecc.

Quindi col piffero che ne bastano 6!

How many of such correspondences would we need to compute both intrinsics and extrinsics? Our projection matrix has 11 degrees of freedom (5 for the intrinsics and 6 for the extrinsics), so we need at least 11 equations. Each correspondence gives us two equations (one for x and y each). Therefore, we would need at least 6 correspondences.

Calibration

UNIVERSITÀ DI PARMA



- Practically we need more than 6 correspondences!
- Consider error in measurements + digitalization error!

Courtesy: Silvio Savarese

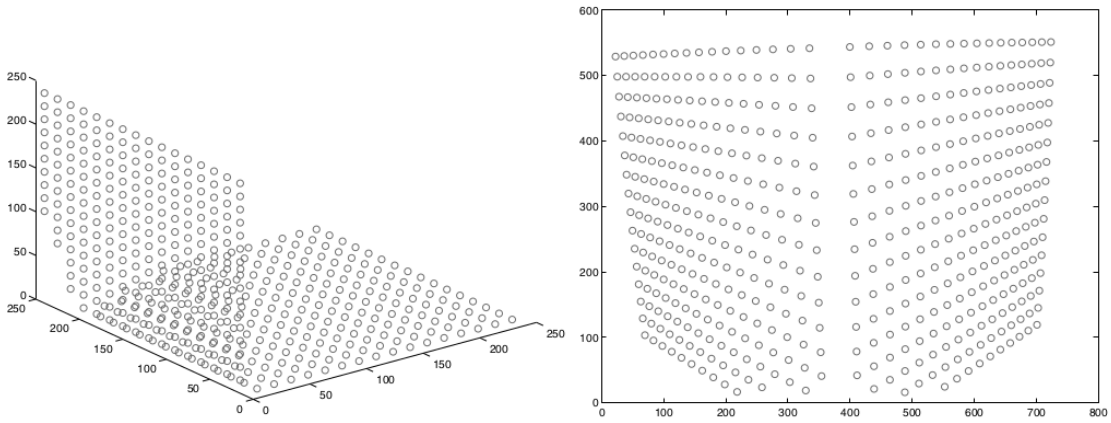
As always, our correspondences are not perfect and susceptible to noise. Having more than the minimal number of correspondences allows us to be more robust to these imprecisions.

Tanti piú punti abbiamo tanto meglio è...

Come impostiamo il sistema?

Example of calibration data

UNIVERSITÀ DI PARMA



- Example of 491 3D points
  - From: Janne Heikkilä; data copyright 2000, University of Oulu.

Esempio di set reale di calibrazione.
Ovviamente mica li hanno presi a manina…

DLT Algorithm

UNIVERSITÀ DI PARMA

- 12 elements but actually there is a dependence among them → 11 unknowns

$$p = K\left[R\,T\right]P = MP$$

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

Ma torniamo a noi.
Date X Y e Z le coordinate mondo e u e v le coordinate immagine, questa è la mia relazione. Attenzione che le mie incognite sono i vari m_{xy}.
Sarebbero 12 ma se vi ricordate come li abbiamo creati in realtà i parametri sotto sono 11 (ovvero c'è interdipendenza tra i vari m). Per ora trascuriamo comunque la cosa.

DLT Algorithm

UNIVERSITÀ DI PARMA

- Direct Linear Transform: rewrite the perspective projection equation as homogeneous system of linear equations

$$p = K\left[R\,T\right]P = MP$$

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

In pratica rappresento queste equazioni come sistema lineare omogeneo.

Calibration

UNIVERSITÀ DI PARMA

$$u_i = \frac{m_{00} X_i + m_{01} Y_i + m_{02} Z_i + m_{03}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}} \qquad v_i = \frac{m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}}$$

$$m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23} - m_{00} u_i X_i - m_{01} u_i Y_i - m_{02} u_i Z_i - m_{03} u_i = 0$$

$$m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13} - m_{20} v_i X_i - m_{21} v_i Y_i - m_{22} v_i Z_i - m_{23} v_i = 0$$

We can recover 2 equations for each correspondence
world ↔ image

Per l'i-eseima corrispondenza noi possiamo ricavare questa coppia di equazioni (spero di non aver fatto casino coi vari indici...)

Calibration

UNIVERSITÀ DI PARMA

Homogeneous System

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 & -v_1 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n & -v_n \end{bmatrix} \begin{bmatrix} m_{00} \\ m_{01} \\ m_{02} \\ m_{03} \\ m_{10} \\ m_{11} \\ m_{12} \\ m_{13} \\ m_{20} \\ m_{21} \\ m_{22} \\ m_{23} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Mettiamo insieme tutte le equazioni dal punto 1 al punto n. Anzi scriviamole in formato matriciale.

È un sistema omogeneo. Non centrano le coordinate omogenee, è che c'è 0 a destra dopo l'=, ovvero I termini noti sono nulli.

Ad essere pignoli è un "sistema lineare omogeneo"

Risolviamolo. Tenendo conto che abbiamo n equazioni e 12 incognite con n >> 12 ed escludendo la soluzione degenere sempre presente in cui tutti gli m sono pari a 0…

Nelle slide successive lo scriveremo in forma semplificata. Non le commento in quanto sono passaggi relativamente ovvi.

Calibration

UNIVERSITÀ DI PARMA

- Step back and write in a simpler form

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_0 \\ m_1 \\ m_2 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_0 \\ m_1 \\ m_2 \end{bmatrix} P_i$$

Gli m con singolo pedice sono quindi una riga 1x4

Calibration

UNIVERSITÀ DI PARMA

- Step back and write in a simpler form

$$u_i = \frac{m_{00} X_i + m_{01} Y_i + m_{02} Z_i + m_{03}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}} \qquad v_i = \frac{m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}}$$

$$u_i = \frac{m_0 P_i}{m_2 P_i} \qquad v_i = \frac{m_1 P_i}{m_2 P_i}$$

$$-m_0 P_i + u_i (m_2 P_i) = 0 \qquad -m_1 P_i + v_i (m_2 P_i) = 0$$

Calibration

UNIVERSITÀ DI PARMA

$$-m_0 P_i + u_i (m_2 P_i) = 0 \qquad -m_1 P_i + v_i (m_2 P_i) = 0$$

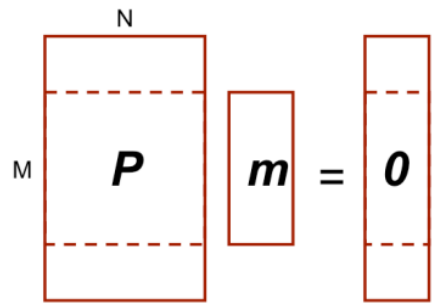
$$\begin{bmatrix} -P_0^T & 0^T & u_0 P_0^T \\ 0^T & -P_0^T & v_0 P_0^T \\ \dots & \dots & \dots \\ -P_n^T & 0^T & u_n P_n^T \\ 0^T & -P_n^T & v_n P_n^T \end{bmatrix}_{2n \times 12} \begin{bmatrix} m_0^T \\ m_1^T \\ m_2^T \end{bmatrix}_{12 \times 1} = \begin{bmatrix} 0 \\ \dots \\ 0 \end{bmatrix} \Rightarrow Pm = 0$$

Homogeneous System → 2n equations

Unknowns → m_i

Calibration

UNIVERSITÀ DI PARMA



- M>N as we suggested
  - Rectangular system of equations
  - Overdetermined
    - "potentially" no solution other trivial one m=0
- The idea is to find the best fitting solution
- Minimize $|Pm|^2$
  - Assuming m != 0 $\longrightarrow |m|^2 = 1$

Courtesy: Silvio Savarese

Il fatto che il numero delle equazioni sia mooolto piú alto del numero delle incognire lo si vede dalle dimensioni della matrice P. M >> N

Le linee tratteggiate indicano il sistema potenzialmente "determinato" con M=N (ignoriamole)

Quindi il sistema potrebbe essere non determinato e ammettere la sola soluzione degenere in cui tutti gli m sono 0. Anzi sarà sicuramente così visto che facciamo errori di misura…

Quindi col fischio che trovo un set di m che mi risolve quelle equazioni.

Ci accontenteremo di trovare il set di m che minimizza Pm assumendo m non nulli.

In generale basta l'assunzione che la norma di m sia diversa da 0 ma si usa quasi sempre 1

Poi c'è un altro problemino. Se trovo una soluzione m anche km (con k scalare) è una soluzione. Ecco qui c'è la fregatura. Io riesco a calibrare al netto di un fattore di scala (cosa che ha però simpatiche applicazioni in ambito cinematografico)

Recap: inverting a non-square matrix

UNIVERSITÀ DI PARMA

- A linear equation

$$Ax = y$$

- Can be solved finding a matrix B (ideally $B=A^{-1}$, namely $AB=I$)

$$x = By$$

- When A is taller than it is wide → possibly no solutions
- When A is wider than it is tall → multiple solutions

Piccolo excursus di recap

Data l'equazione lineare generica io dovrei invertire la A
Se A è rettangolare capite che non ci riesco. Per cui ha senso parlare di pseudoinversa.

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- Partially generalize matrix inversion for non square matrices
  - Do you remember something?
- Each matrix can be factorized into **singular vectors** and **singular values**
- More precisely each matrix A can be seen as product of three matrices:

$$A = UDV^T$$

E come la trovo questa pseudoinversa?
Ogni matrice può essere fattorizzata in un prodotto di vettori singolari e valori singolari. C'è un legame con autovettori e autovalori ma lascio a voi approfondire.

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- Supposing A as $m \times n$ matrix, U is a $m \times m$ matrix, D is a $m \times n$ one and V is a $n \times n$ one
- U and D orthogonal matrices
  - Namely $UU^T=U^TU=I$ and $VV^T=V^TV=I$
  - Columns are the left (U) or right (V) singular vectors
- D is a diagonal matrix
  - Elements along the diagonal are the singular values

Una matrice è ortogonale quando colonne e righe sono mutualmente ortonormali ovvero $x^Ty=0$

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- A linear equation

$$Ax = y$$

- Can be solved finding a matrix B (ideally $B=A^{-1}$)

$$x = By$$

- When A is taller than it is wide → possibly no solutions
- When A is wider than it is tall → multiple solutions

Torniamo qui. Come la trovo la matrice pseudoinversa di A?

Moore-Penrose pseudoinverse

UNIVERSITÀ DI PARMA

- The pseudoinverse of a matrix A or A^+ is defined as

$$A^+ = \lim_{\alpha \to 0} \left(A^T A + \alpha I \right)^{-1} A^T$$

- Practically A^+ can be approximated using SVD

$$A^+ = V D^+ U^T$$

- When A is taller than it is wide this allows to compute the best approximation

D+ lo ottengo prendendo i reciproci dei valori della diagonale di D e calcolando la trasposta

Calibration

UNIVERSITÀ DI PARMA

- How we can solve such system?

$$\mathrm{P}m=0$$

- Via SVD decomposition!
  - This leads to the optimal solution assuming $|m|^2=1$

Una cosa molto simile la applico al nostro caso. Come minimizzo Pm?

Rammentare che OpenCV fornisce già SVD oltre a quasi tutte le librerie matematiche del mondo

Recall that for homogeneous linear system such as Pm = 0, the SVD gives us the least squares solution, subject to |m| = 1.

Calibration

UNIVERSITÀ DI PARMA

$$\mathbf{P}\,\mathbf{m} = 0$$

SVD decomposition of P

$$\mathbf{U}_{2n\times 12}\,\mathbf{D}_{12\times 12}\,\mathbf{V}^{\mathrm{T}}{}_{12\times 12}$$

- The last V column gives us m

$$m \overset{\text{def}}{=} \begin{pmatrix} \mathbf{m}_1^{\mathrm{T}} \\ \mathbf{m}_2^{\mathrm{T}} \\ \mathbf{m}_3^{\mathrm{T}} \end{pmatrix} \Rightarrow M$$

Courtesy: Silvio Savarese

We can obtain the least-squares solution for m by factorizing P to UDV^T using SVD and then taking the last column of V.

Once m is computed, **we can repackage it into M** and compute all the camera parameters as we'll see next;

U -> matrice ortogonale (una sorta di rotazione)
D -> matrice diagonale (quindi elementi non nulli solo sulla diagonale e in ordine decrescente il più grande in alto a sinistra fino a scendere (anche a 0))
V -> matrice ortogonale

OpenCV fornisce SVD (che comunque vale per qualunque matrice) con una singola chiamata.

m comunque sono i 12 elementi di M grande e rimane il miglior possibile che otteniamo.

Ci basta? Boh posso applicarlo alla trasformazione e vedere se ottengo i punti del mondo misurati. Se il risultato fa schifo o i punti di partenza non vanno bene o praticamente sempre ho cannato le misure.

Parameters Extraction

UNIVERSITÀ DI PARMA

Courtesy: Silvio Savarese

$$M = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} \rho$$

- We can recover parameters but with a scale factor ρ
- Actually not an unknown, we can impose $|m|=1$
  - or equivalent Froebius norm $||M||=1$
- Extrinsic and Intrinsic parameters are separately extracted

Applicato quanto visto abbiamo ricavato I vari m. Io voglio I parametri che mi portano a questi m (anche perché vorrei per lo meno dividere tra estrinseci e intrinseci.

Estraiamo un rho di scala.

Once we have estimated the **combined** parameters in m, the next step is to extract the **actual intrinsics and extrinsics** from it. Note that M has 11 degrees of freedom and can be computed up to a scale parameter. We explicitly identify the scale parameter as ρ. Notice that ρ it's not a real unknown. We can estimate ρ by using the fact that |m| must be =1 (or, equivalently, the Frobenius norm of M = 1).

Parameters Extraction

UNIVERSITÀ DI PARMA

$$\frac{M}{\rho} = \begin{pmatrix} \alpha \mathbf{r}_1^T - \alpha \cot\theta \mathbf{r}_2^T + u_0 \mathbf{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta}\mathbf{r}_2^T + v_0 \mathbf{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \mathbf{r}_3^T & t_z \end{pmatrix} = \frac{K}{\rho}\begin{bmatrix} R & T \end{bmatrix}$$

$$K = \begin{bmatrix} \alpha & -\alpha\cot\theta & u_o \\ 0 & \frac{\beta}{\sin\theta} & v_o \\ 0 & 0 & 1 \end{bmatrix}$$

$A_{3\times3}$ $\quad$ $\mathbf{b}_{1\times3}$

Box 1

$$A = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \mathbf{a}_3^T \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Estimated values

Intrinsic

$$\rho = \frac{\pm 1}{|\mathbf{a}_3|} \qquad u_o = \rho^2(\mathbf{a}_1 \cdot \mathbf{a}_3)$$

$$v_o = \rho^2(\mathbf{a}_2 \cdot \mathbf{a}_3)$$

$$\cos\theta = \frac{(\mathbf{a}_1 \times \mathbf{a}_3)\cdot(\mathbf{a}_2 \times \mathbf{a}_3)}{|\mathbf{a}_1 \times \mathbf{a}_3| \cdot |\mathbf{a}_2 \times \mathbf{a}_3|}$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

It is relatively straigthforward to derive all the intrinsics and extrinsics from the 3x4 matrix M (whose entries we estimated in the previous slides).

We refer to [FP], Sec. 1.3.1, for the details or leave it as an exercise.

To avoid overcomplicating the notation, we rename M/ρ as [A b], and provide a solution for the intrinsics and extrinsics as function of the elements of A and b (as defined in the box 1 in the slide). A solution for the scale ρ, offset and skew are reported in the slide

A è una matrice 3x3, di fatto è la moltiplicazione di K e R
B è KT quindi vettore colonna di 3 elementi

Alla finme ricavo T e R e data R ricavoi i 3 angoli che l'hanno generata (non ci sono i passaggi)

Parameters Extraction

UNIVERSITÀ DI PARMA

$$\frac{\mathcal{M}}{\rho} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} = \mathrm{K}[\mathrm{R} \quad \mathrm{T}]$$

A b

$$\mathrm{A} = \begin{bmatrix} \mathbf{a}_1^{\mathrm{T}} \\ \mathbf{a}_2^{\mathrm{T}} \\ \mathbf{a}_3^{\mathrm{T}} \end{bmatrix} \quad \mathrm{b} = \begin{bmatrix} \mathrm{b}_1 \\ \mathrm{b}_2 \\ \mathrm{b}_3 \end{bmatrix}$$

Estimated values

Intrinsic

$$\alpha = \rho^2 |\mathbf{a}_1 \times \mathbf{a}_3| \sin\theta$$

$$\beta = \rho^2 |\mathbf{a}_2 \times \mathbf{a}_3| \sin\theta$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Here we have solution for alpha and beta and, thus, for the focal length.

A è una matrice 3x3, di fatto è la moltiplicazione di K e R
B è KT quindi vettore colonna di 3 elementi

Alla finme ricavo T e R e data R ricavoi i 3 angoli che l'hanno generata (non ci sono i passaggi)

Parameters Extraction

UNIVERSITÀ DI PARMA

$$\frac{\mathcal{M}}{\rho} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} = \mathrm{K}[\mathrm{R} \quad \mathrm{T}]$$

A b

$$\mathrm{A} = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \mathbf{a}_3^T \end{bmatrix} \qquad \mathrm{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Estimated values

Extrinsic

$$\mathbf{r}_1 = \frac{(\mathbf{a}_2 \times \mathbf{a}_3)}{|\mathbf{a}_2 \times \mathbf{a}_3|} \qquad \mathbf{r}_3 = \frac{\pm \mathbf{a}_3}{|\mathbf{a}_3|}$$

$$\mathbf{r}_2 = \mathbf{r}_3 \times \mathbf{r}_1 \qquad \mathrm{T} = \rho \, \mathrm{K}^{-1} \mathbf{b}$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Here we have solu7ons for the extrinsics R and T.

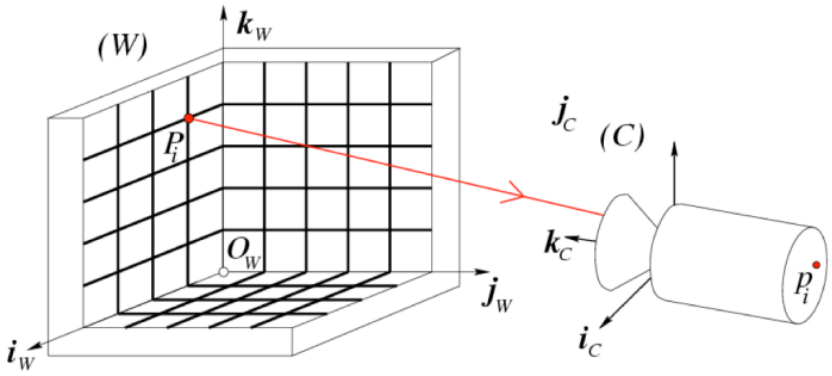
A è una matrice 3x3, di fatto è la moltiplicazione di K e R
B è KT quindi vettore colonna di 3 elementi

Alla finme ricavo T e R e data R ricavoi i 3 angoli che l'hanno generata (non ci sono i passaggi)

Rammentare che non è come si fa normalmente. In genere non si fa in un colpo solo ma si estraggono gli intrinseci slegati dalla posizione della camera (**tanto sono quelli**). Poi di volta in volta calcolo gli estrinseci (fare esempio automotive).

Degenerate Case

UNIVERSITÀ DI PARMA



- Not all P_ws lead to a result!
  - P_w points must not lie on the same plane
  - P_w points must not lie on the intersection curve of 2 quadric surfaces

Courtesy: Silvio Savarese

Is the homogenous system introduced in Eq 4 solvable regardless of how points Pi are distributed in 3D? No…. There are certain configurations that won't allow the system to be solved – these are called degenerate configurations. For instance, Pi's cannot lie on the same plane. Or points cannot lie on the intersection curve of two quadric surfaces (see [FP] section 1.3, page 25)

Rammentare ancora che OpenCV fornisce pure tool di calibrazione

UNIVERSITÀ DI PARMA

Calibration
Zhang approach (2000)

Approccio di Zhang

Zhang approach (2000)

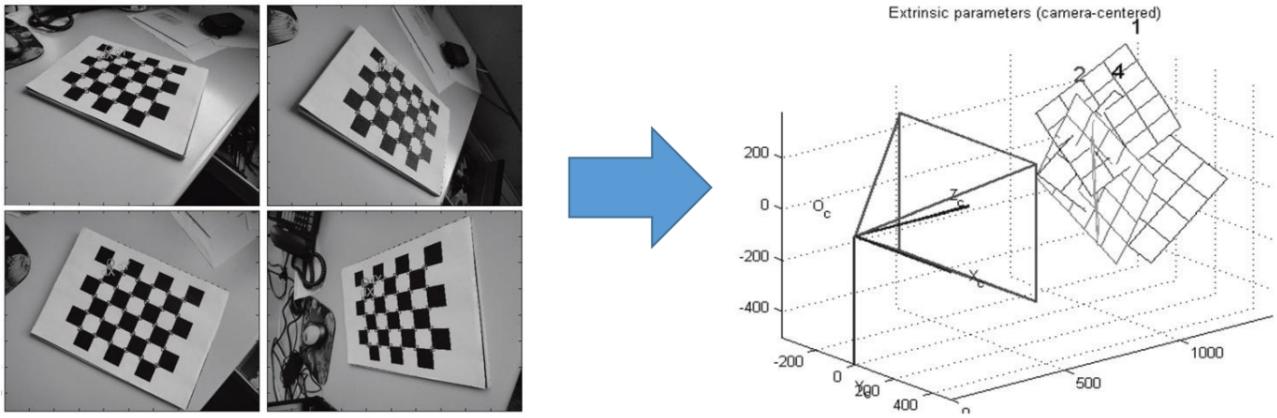
UNIVERSITÀ DI PARMA

- Not very practical to use 3D calibration rigs…
- Usually calibration grids are “flat” (Matlab, OpenCV)
- This does not fit well with Tsai’s requirements
  - No coplanar points
- Zhang’s approach, conversely, exploits this!

La griglia di Tsaj è scomodina…
Ci piacerebbe tanto calibrare usando una griglia planare…
Esattamente quanto Tsaj dice che non va bene…

Zhang approach (2000)

UNIVERSITÀ DI PARMA

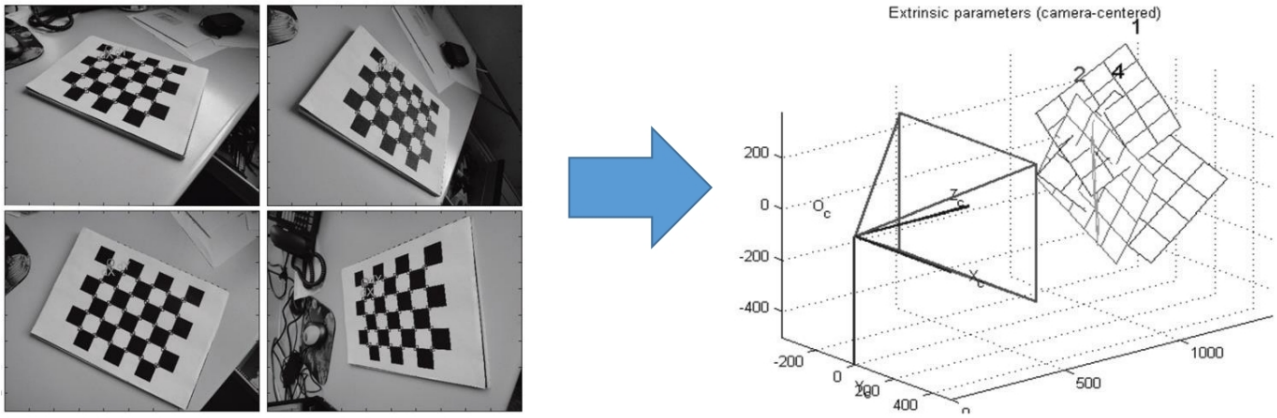


Zhang, *A flexible new technique for camera calibration*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000.

L’approccio di Tsaj sfrutta griglie poste di fronte alla nostra telecamera in posizione abbastanza libera. Comodo! Però recupereremo solo gli intrinseci sostanzialmente (ma sono quelli che ci servono di piú!)

Zhang approach (2000)

UNIVERSITÀ DI PARMA

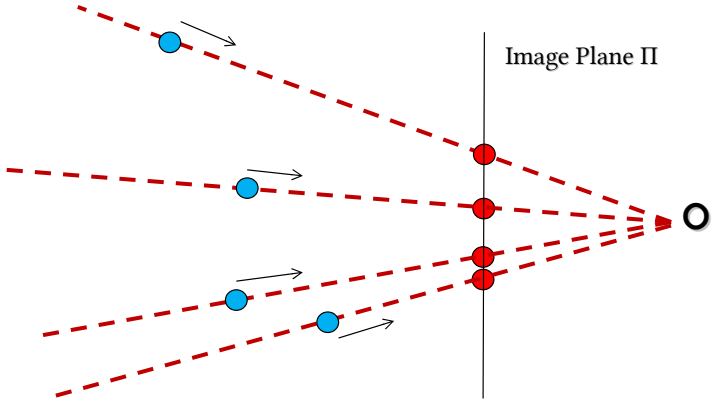


Before detailing Zhang's approach, just understand what does it mean to frame a "plane"

L'approccio di Tsaj sfrutta griglie poste di fronte alla nostra telecamera in posizione abbastanza libera. Cvomodo! Però recupereremo gli intrinseci sostanzialmente (ma sono quelli che ci servono di piú!)

Ill posed problem

UNIVERSITÀ DI PARMA



- Projective space ⟶ we lost one dimension
- 3D [x,y,z] are projected in [x/z,y/z,1]

Anticipo queste slide che riprenderemo la lezione successiva (che è un bagno di sangue!)

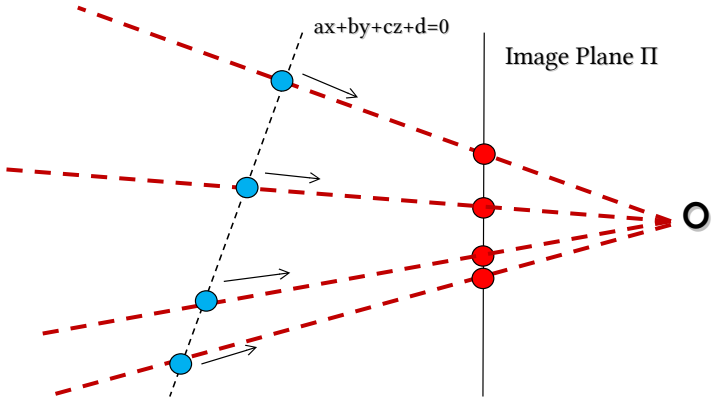
Ho ricavato quanto lega mondo e punto immagine.

Tutto risolto? Sì fintantoché il mondo è noto e fintantoché noi vogliamo passare dal mondo all'immagine. Il contrario mica tanto…
È una proiettività passiamo da R3 a R2

Once the camera is calibrated (intrinsics are known) and the transformation from the world reference system to the camera reference system (which accounts for the extrinsics) is also known, can we estimate the location of a point P in 3D from its (known) observation p? The general answer to this question is no. This is because, given the observation p, even when the camera intrinsics and extrinsic are known, **the only thing we can say is that the point P is located somewhere along the line defined by C** and p. This line is called the line of sight. The actual location of P along this line is unknown and cannot be determined from the observation p

Add a constraint

UNIVERSITÀ DI PARMA



- Just image that the (red) image points are projection of points that lie on the same plane
- In such a case for each image point there is only one world point (cyan)

Si risolverebbe con stereo… ma lo vedremo piú avanti.

Per ora ce lo tiriamo dietro. Ma posso aggiungere informazione?

Ad esempio se so che i punti sono allineati qualcosa si può fare

Ad esempio se inquadro qualcosa che è su un piano?

Quindi se aggiungo vincolo qualcosa trovo. Se nel nostro mondo so che ci sono dei piani allora posso fare qualcosa…

Add a constraint

UNIVERSITÀ DI PARMA

- Given a plane aX + bY + cZ + d = 0

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} \overset{3\times4}{=} M \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} u \\ v \\ w \\ 0 \end{bmatrix} = \overset{4\times4}{\begin{bmatrix} M \\ a\,b\,c\,d \end{bmatrix}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = \begin{bmatrix} M \\ a\,b\,c\,d \end{bmatrix}^{-1} \begin{bmatrix} u \\ v \\ 1 \\ 0 \end{bmatrix} \xrightarrow[\text{euclidean}]{} \begin{bmatrix} X/W \\ Y/W \\ Z/W \end{bmatrix}$$

- We can obtain euclidean world coordinates of pixel (u,v)

Vi ricordate vero che M è 4x3?

Aggiungiamo un vincolo, ovvero che il punto mondo sia su un piano con quella equazione (sì, quella è l'equazione di un piano nel mondo 3D)

In pratica aggiungo una riga alla mia matrice e uno 0 sotto le coordinate omogenee immagine. Questo 0 mi impone che il punto 3D sia su quel piano (=0)

Oppalà, la matrice è 4x4. Ma quindi ora riesco a invertirla! (ok non fate I precisini, ovviamente deve aver det != 0, ma sarà così in generale)

Quindi se conosco un piano nel mondo di cui conmosco i parametri riesco a fare la ricostruzione inversa. Ovviamente solo per i punti del piano…

Specific case, plane Z=0

UNIVERSITÀ DI PARMA

- Consider a very specific plane → Z=0

$$\begin{bmatrix} u \\ v \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} \qquad \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = H \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

- H is 3×3
- H is an **Homography**
- This actually works for every possible plane!

Consideriamo un piano specifico z=0

Ho uno 0 che moltiplica la terza colonna → eliminamola!

Ma non ci serve neanche l'ultima riga visto che conosco già il piano, togliamo anche lei! (comunque sarebbe [0 0 1 0])

Riduciamo il tutto a una matrice 3x3

H in quanto OMOGRAFIA (ne parleremo… ma in pratica la proiezione su di un piano)

Better understanding H

UNIVERSITÀ DI PARMA

- We started from a general M but
  - M = K [R T]

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} \alpha & -\alpha\cot\theta & c_x \\ 0 & \frac{\beta}{\sin\theta} & c_y \\ 0 & 0 & 1 \end{bmatrix}}^{K_{3\times3}} \overbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}}^{[RT]_{3\times4}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Invece di semplificare la M vediamo se riesco a semplificare i suoi “componenti”
Esplicitiamo K e [RT]

Better understanding H

UNIVERSITÀ DI PARMA

- When the Z = 0 we can simplify as:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ r_{31} & r_{32} & t_z \end{bmatrix}_{3\times 3} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Applichiamo quanto visto nel caso del piano z=0
Andiamo dritti al punto, questo mi semplifica la [RT] eliminando una colonna

Better understanding H

UNIVERSITÀ DI PARMA

- When the Z = 0 we can simplify as:

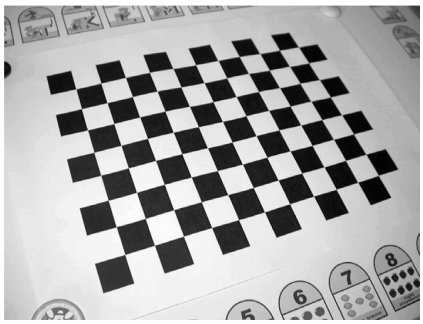
$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ r_{31} & r_{32} & t_z \end{bmatrix}_{3\times 3} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \overbrace{K \begin{bmatrix} r_1 r_2 t \end{bmatrix}}^{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Senza tirarci dietro tutta la matrice la posso scrivere così

Zhang's approach

UNIVERSITÀ DI PARMA

- Zhang idea is to use a "planar" calibration checkboard
  - Set the world coordinate system onto the checkboard ⟶ z = 0 plane…
- In such a case we can apply an homography!



Ma se quello è il mio piano z=0 applico quanto visto poco prima

Calibration

UNIVERSITÀ DI PARMA

- For each point on the planar surface of my grid I then obtain
- In the same image…

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = H \begin{bmatrix} X_i \\ Y_i \\ 1 \end{bmatrix} \quad \text{with} \quad i = 1, 2, 3, \ldots, n$$

Come visto in precedenza per Tsaj uso tanti punti e per ciascuno ho quella relazione

Calibration

UNIVERSITÀ DI PARMA

- Again a system of *n* equations
  - Unknowns are H elements

$$QH = 0$$

Come prima raccolgo i vari elementi e arrivo ad un sistema omogeneo (vi risparmio i passaggi)

Calibration

UNIVERSITÀ DI PARMA

- For solving this issue we can use the same approach used for “Tsai”
- We have a 3×3 homograpy instead of a 3×4 projection matrix
- We need to solve QH = 0 where Q is 2n×9 and has rank 8
- We need at least 4 points
  - When n > 4 ⟶ SVD

Il sistema diventa QH = 0 dove Q è 2n * 9 e ha rango 8

Calibration

UNIVERSITÀ DI PARMA

- For computing H we can, again, use a SVD
- It is, again, an homogeneous system

$$QH = 0$$

- Is H really useful?
  - No, given they mix intrinsic parameters and very specific extrinsic ones
  - I put my coords system on the grid!

Quindi mi ricavo H. Bellino ma che mi serve? Tipicamente a nulla salvo che non serva esattamente per quallo specifico piano su cui ho la griglia di calibrazione (cioé mai)

Infatti gli estrinseci sono specifici di come ho messo la griglia (aka non mi servono)

Ma la componente K mi servirebbe… e se ho usato tante griglie ho ricavato tante H differenti, ciascuna delle quali però deriva dalla stessa K

Computing H

UNIVERSITÀ DI PARMA

- Each view has different H but **K is the same for all views**

$$H^i = K\left[r_1^i \; r_2^i \; t\right]$$

- Once computed H^i we can focus on $H^i \leftrightarrow K$ relation

Non entrerei nei dettagli ma in pratica il fatto che gli r siano ortonormali significa ($r_1^T r_2=0$ $||r_1||=||r_2||=1$) e quindi lo sfruttiamo:

$r_1=K^{-1}h_1$ e $r_2=K^{-1}h_2$ se vale $r_1^T r_2=0 \longrightarrow h_1^T K^{-T} K^{-1} h_2=0$

Computing K given H

UNIVERSITÀ DI PARMA

- K can be anyway computed from H
  - Constraints: r_1, r_2 form an orthonormal basis
    - This allows to compute$K^{-T}K^{-1}$
  - Cholesky decomposition can be applied to the symmetric and positive $B=K^{-T}K^{-1}$ for computing K^T
  - Constraints: we need multiple views
    - We will discuss this later (without details)

Non entrerei nei dettagli ma in pratica il fatto che gli r siano ortonormali significa ($r_1^T r_2=0$ $||r_1||=||r_2||=1$) e quindi lo sfruttiamo:

$r_1=K^{-1}h_1$ e $r_2=K^{-1}h_2$ se vale $r_1^T r_2=0 \longrightarrow h_1^T K^{-T}K^{-1}h_2=0$

Problema: ci riesco solo se inquadro piú griglie (che poi non siano mere traslazioni o rotazioni)

Computing extrinsic parameters

UNIVERSITÀ DI PARMA

- We now have multiple H and one K
- We can compute extrinsic parameters of each view from them…
  - A scale factor is, again, to be considered…
  - λ

$$
\begin{aligned}
r_1 &= \lambda K^{-1} h_1 \\
r_2 &= \lambda K^{-1} h_2 \\
r_3 &= r_1 \times r_2 \\
t &= \lambda K^{-1} h_3
\end{aligned}
$$

E se ci servono estrinseci specifici? Estremizzando, una delle griglie la piazzo lì (ma non è il metodo usato)
Comunque in generale gli estrinseci sono molto piú facili e meno soggetti ad errore da misurarsi
Il metodo tipico è quello della Perspective from *n* Points

How many grids?

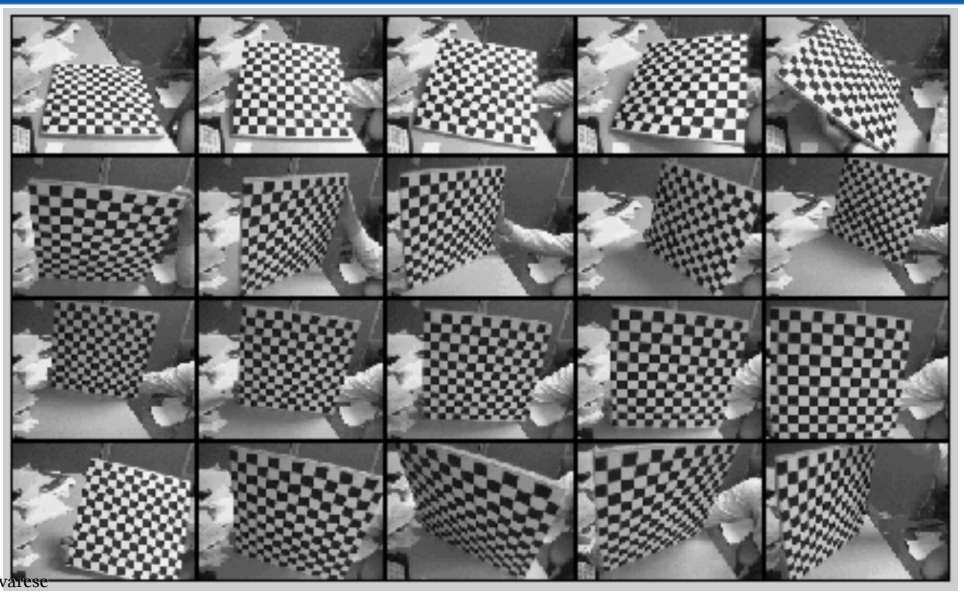
UNIVERSITÀ DI PARMA

- With a “single” H we can not anyway fully compute K
- 3 Hs are needed, at least, to compute K
  - The larger the number of views, the better the result
  - Usually 20-50 grid views

Come accennato da una sola H non riesco a ricavare un K univoco

Example of calibration data

UNIVERSITÀ DI PARMA

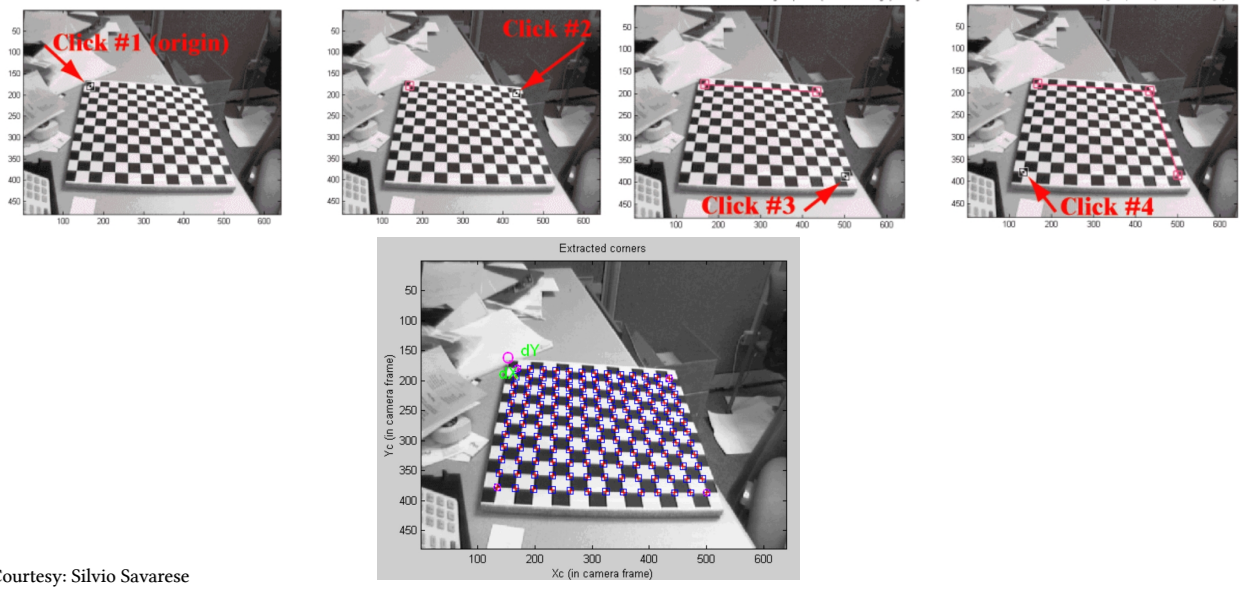


Courtesy: Silvio Savarese

Passo 1 acquisisco differenti immagini di opportuna griglia

Example of calibration data

UNIVERSITÀ DI PARMA



Courtesy: Silvio Savarese

Passo 2 seleziono corner
Passo 3 il sistema me li trova automaticamente

Questo è un sistema semi-automatico. All'atto pratico si usano diffusamente sistemi del tutto automatici e/o al limite supervisionati

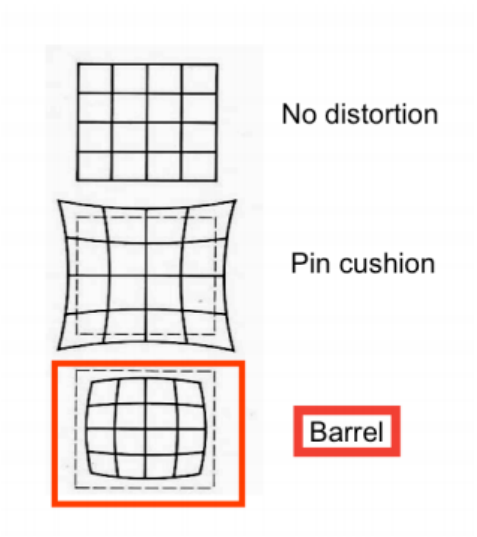
UNIVERSITÀ DI PARMA

Lens Distortion

LENS DISTORTION

Lens Distortion

UNIVERSITÀ DI PARMA



Courtesy: Silvio Savarese

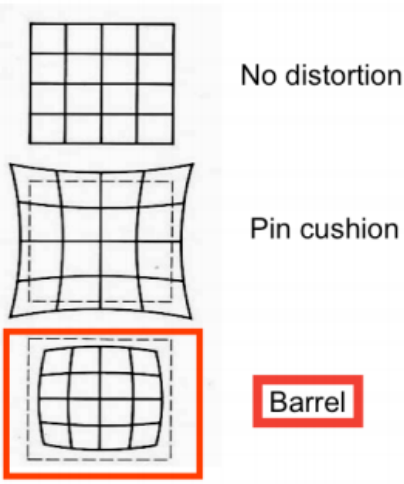
Nell'esempio si mostra distorsione solo radiale ma i casi sono ancora piú complessi (ad esempio telecamera dietro un parabrezza curvo…)

So far, we have been working with ideal lenses which are free from any distortion. However, real lenses can deviate from rectilinear projection. The resulting distortions are ojen radially symmetric, which can be aeributed to the physical symmetry of the lens. The image above exhibits barrel distortion. This occurs when the image magnification decreases with distance from the optical axis. Fisheye lenses usually produce this type of distortion.

Radial Lens Distortion

UNIVERSITÀ DI PARMA

- We assumed a “thin” lens
  - This basically means that lines are lines
  - Unfortunately this is not true!
- Radial Distortion
  - Image magnification/contraction depends on distance from optical axis
  - Deviations are more evident on lateral areas
  - Cheap optics



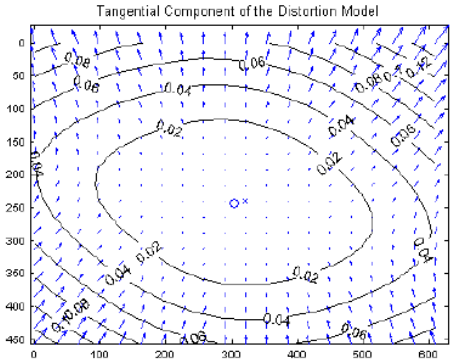
Courtesy: Silvio Savarese

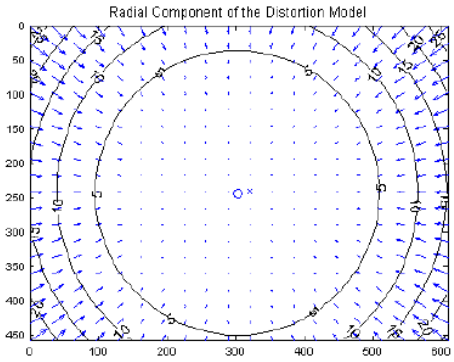
Ci limiteremo solo alla distorsione radiale…
Rammentare ancora il discorso delle lenti imperfette

Tangental Lens Distortion

UNIVERSITÀ DI PARMA

- Typical of misaligned lenses
  - Not parallel to image sensor



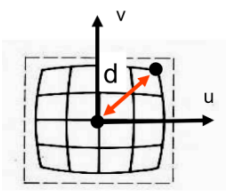


Courtesy: Technische Universität München

Ci limiteremo solo alla distorsione radiale…

Radial Lens Distortion

UNIVERSITÀ DI PARMA



$$S_\lambda \begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad M\,P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i$$

Distortion coefficient

$$\lambda = 1 \pm \sum_{p=1}^{3} \kappa_p d^{2p} \qquad d^2 = a\,u^2 + b\,v^2 + c\,u\,v$$

[Eq. 5] Polynomial function — To model radial behavior [Eq. 6]

- λ is the distortion factor
  - Actually it is a $\lambda_{(u,v)}$ since it depends on the distance from optical axis

Courtesy: Silvio Savarese

La matrice sarebbe di per sé quella di un fattore di scala. Peccato che lambda non sia costante.

L'equazione 5 la possiamo usare per modellizzare come lambda cambia, è una espansione polinomiale. È comunque un compromesso tra precisione del modello e complicazione della risoluzione si può anche andare sul primo grado per lenti poco distorcenti. I vari kp sono comunque piccoli
Rammentiamo che la distanza dal centro dell'immagine la calcolo in base a d2=u2+v2=||K^-1p||-1
In pratica dipende da d che a sua volta è esprimibile come funzione delle coordinate (u,v)

Rammentare che introduco una non linearità.

The radial distortion can be modeled using an isotropic transformation Sλ as shown in the slide. This transformation is regulated by the distortion factor λ.

Notice that unlike a traditional scale transformation (of the type [s 0 0;0 s 0; 0 0 1]), where s is constant, λ is a function of the distance d from the center (since the distortion is radially symmetric about the center) and thus function of the coordinates ui, vi .

This causes the distortion transformation to introduce a non-linearity to the mapping from Pi to pi

We approximate λ using a polynomial expansion (Eq. 5). The resulting coefficients, κp, are known as the distorton coefficients. In order to model the radial behavior, the distance can be expressed as quadratic function of the coordinates u,v in the image plane (Eq. 6).

Radial Lens Distortion

UNIVERSITÀ DI PARMA

$$\underbrace{\begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{Q} M P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i \qquad Q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

- λ depends on u,v and this leads to a non linearity in the P ⟶ p mapping

Courtesy: Silvio Savarese

Notice that unlike a traditional scale transformation (of the type [s 0 0;0 s 0; 0 0 1]), where s is constant, λ is a function of the distance d from the center (since the distortion is radially symmetric about the center) and thus function of the coordinates ui, vi .

This causes the distortion transformation to introduce a non-linearity to the mapping from Pi to pi

Radial Lens Distortion

UNIVERSITÀ DI PARMA

$$\underbrace{\begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{Q} M P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i \qquad Q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

- Tsai 87, initially estimate m_1 and m_2 by SVD and then $m_3=f(m_1, m_2, \lambda)$

Courtesy: Silvio Savarese

F() non è lineare e comunque ci sono condizioni degeneri che impediscono il calcolo di m1 e m2

Alternativa: general calibration spiegato schematicamente nel seguito

Iterative, starts from initial solution
May be slow if initial solution far from real solution
Estimated solution may be function of the initial solution
Newton requires the computation of J, H
Levenberg-Marquardt doesn't require the computation of H

A possible algorithm
1. Solve linear part of the system to find approximated solution
2. Use this solution as initial condition for the full system
3. Solve full system using Newton or L.M.

Typical assumptions:
- zero-skew, square pixel
- u o , v o = known center of the image

UNIVERSITÀ DI PARMA

OpenCV Calibration

OpenCV calibration

Multi plane calibration

UNIVERSITÀ DI PARMA

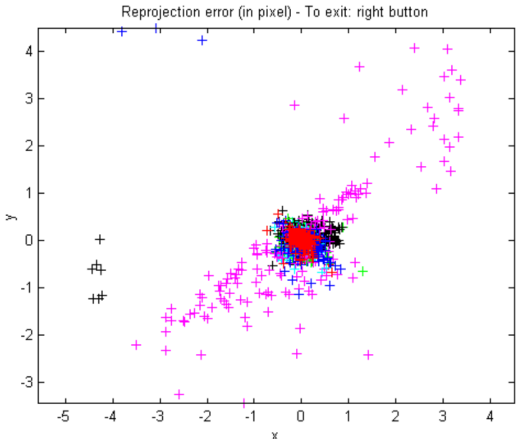
- Requires a plane only!
- No known positions/orientations
- Strobl et al. approach (2011)

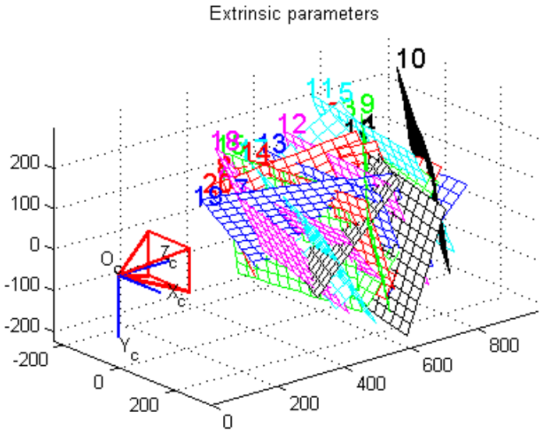
- Free code:
  - https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

Rivisitazione migliorata dell'approccio di Zhang
LE intersezioni dei quadrotti sono ricavate tramite Harris Corner Detection (che vedremo piú avanti)

Example of calibration data

UNIVERSITÀ DI PARMA





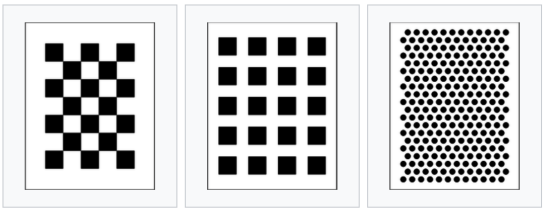
Courtesy: Silvio Savarese

Passo 2 seleziono corner
Passo 3 il sistema me li trova automaticamente

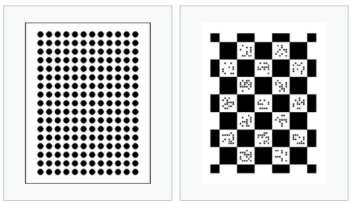
Questo è un sistema semi-automatico. All'atto pratico si usano diffusamente sistemi del tutto automatici e/o al limite supervisionati

Calibration Patterns

UNIVERSITÀ DI PARMA

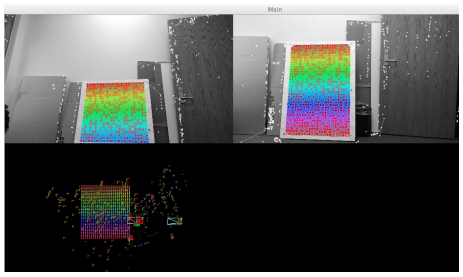


Chessboard | Square Grid | Circle Hexagonal Grid



Circle Regular Grid | ECoCheck

https://boofcv.org/index.php?title=Tutorial_Camera_Calibration



https://github.com/arpg/Documentation/tree/master/Calibration

UNIVERSITÀ DI PARMA

Perspective-n-Point

OpenCV calibration

Extrinsic parameters

UNIVERSITÀ DI PARMA

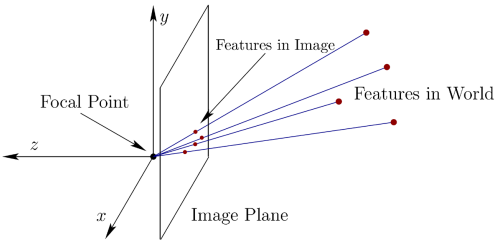
- Tsaj, Zhang, and other methods allow to easily compute K
  - Intrinsic parameters “only”
- General problem:
  - I already computed K
  - I want to compute or update [RT]
  - 6 DOF (3 translation, 3 rotation)
- Namely, camera pose estimation “only”

Tutti i metodi visti mi ricavano bene la K. Ma non [R T].
K è grossomodo fisso e quindi è in effetti il piú importante da ricavarsi.
R e T possono cambiare spesso e quindi oltretutto ci sta doverlo ricalcolare spesso.

Perspective-n-Point

UNIVERSITÀ DI PARMA

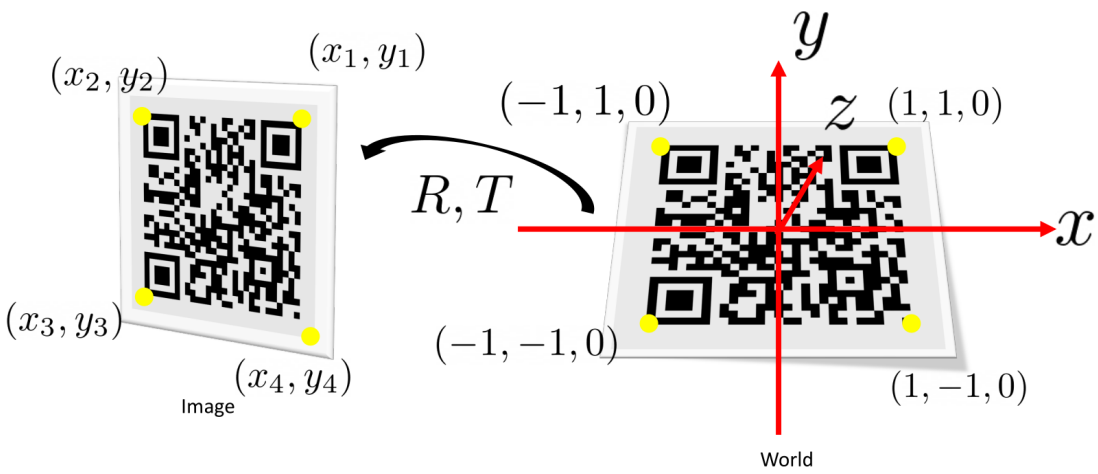
- We know:
  - A set of *n* world coordinates P_w
  - Their projections p_c on an image
  - Camera Intrinsics K
- Unknowns
  - 3D camera rotation wrt world coordinates R
  - 3D camera translation wrt wolrd reference origin
- Different approaches



Ipotizziamo di conoscere n punti nel mondo con le loro coordinate e le relative proiezioni.
Riesco a ricavare l'R e il T della camera? Sì. Anzi è algebricamente risolvibile al netto del rumore

PnP Problem with QR code

UNIVERSITÀ DI PARMA



Courtesy: Yu Xiang

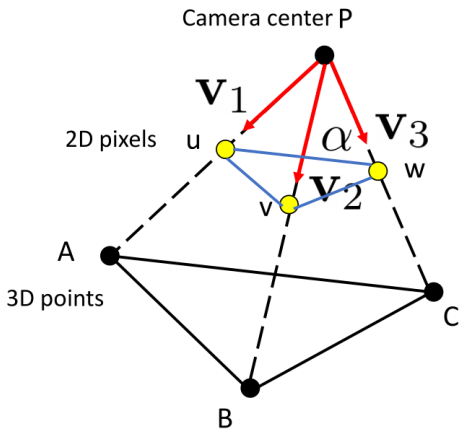
Esempio di applicazione, uso marker specifici che mi permettono di orientarmi

P3P

UNIVERSITÀ DI PARMA

- I can use only 3 known points (A, B, C)
- And their projections u, v, w

$$
\begin{aligned}
u &= K[I\,0]A \\
v &= K[I\,0]B \\
w &= K[I\,0]C
\end{aligned}
$$



Courtesy: Yu Xiang

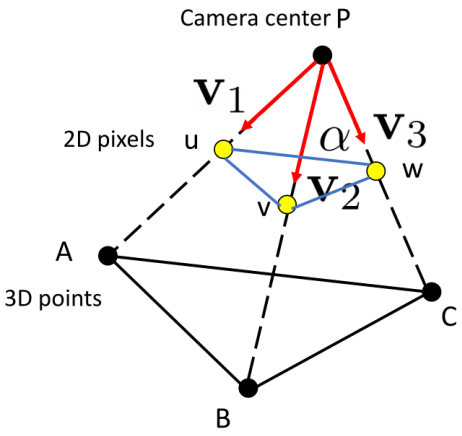
Se uso riferimento su camera quelle sono le relazioni

P3P

UNIVERSITÀ DI PARMA

- K^{-1} can be computed for sure
- If I multiply K^{-1} for u, v, w what is the result?
  - Lines of sight

$$\vec{v}_1 = K^{-1}u$$
$$\vec{v}_2 = K^{-1}v$$
$$\vec{v}_3 = K^{-1}w$$



Courtesy: Yu Xiang

Si può dimostrare che la proiezione sull'immagine moltiplicata per l'inversa della K mi fornisce come risultato un vettore.
Questo vettore mi codifica il punto di vista (sono quelli in rosso)

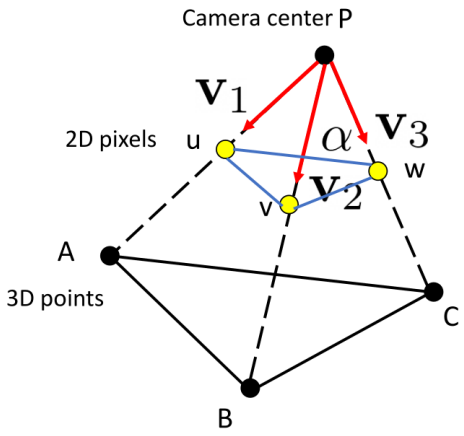
P3P

UNIVERSITÀ DI PARMA

- Using the 3 vectors we can also compute the angles between them
- As example:

$$\cos(\alpha) = \frac{\vec{v}_2 \cdot \vec{v}_3}{\|v_2\| \ \|v_3\|}$$

- The same for other angles (γ and β)



Courtesy: Yu Xiang

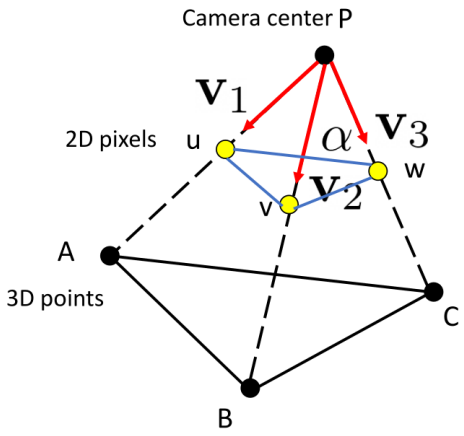
Con questa formula posso calcolare l'angolo tra due vettori. In pratica è il prodotto scalare normalizzato.

P3P

UNIVERSITÀ DI PARMA

- We can compute the distance of points A, B, and C from P exploiting:

$$\overline{AB}^2 = \overline{PA}^2 + \overline{PB}^2 - 2\cos(\gamma)$$
$$\overline{AC}^2 = \overline{PA}^2 + \overline{PC}^2 - 2\cos(\beta)$$
$$\overline{BC}^2 = \overline{PB}^2 + \overline{PC}^2 - 2\cos(\alpha)$$



- PA, PB, and PC are the unknowns!

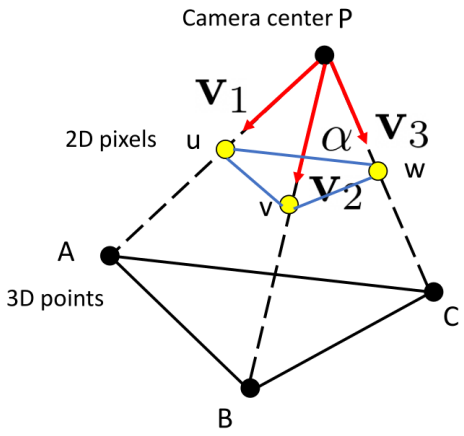
Courtesy: Yu Xiang

La distanza tra P e i punti "mondo" è calcolabile con quella formula.
Ma queste distanze sono proprio quelle che voglio trovare perché mi permettono di stimare come si trova P rispetto a questi punti (in pratica se triangolo rispetto due coppie di punti noti riesco a trovare la mia posizione nel mondo o meglio riesco a trovare 4 posizioni)

P3P

UNIVERSITÀ DI PARMA

- We can solve previous equation system but we obtain 4 possible solutions…
- A 4th point can be used which one aligns most accurately
  - We should rather say P3+1P
  - No exact solution for sure (noise)
- Given the relative position between P and the 3 world points we can recover camera pose



Courtesy: Yu Xiang

Con 3 punti ho 4 possibili soluzioni
Uso un 4 punto per stimare quella giusta
Dico stimare in quanto è sicuro che devo tener conto di rumore…

P3P with RANSAC

UNIVERSITÀ DI PARMA

- RANSAC is a method to estimate parameters from noisy observed data
- Yes, our measurement are for sure noisy!
- Instead of using 3+1 points, use N points where N >> 3
  - Randomly select 3 points
  - Use P3P to compute camera pose ⟶ current hypothesis
  - Count how many points support current hypothesis
  - Go back to point selection until we found a suitable solution or sufficient trials

Se voglio essere piú preciso posso usare tanti punti. Ne pesco 3 alla volta a caso e calcolo un risultato. Uso il risultato per vedere quanto si concilia con gli altri punti.
Ripeto un po' di volte e termino o quando ho trovato un risultato soddisfacente o dopo un numero di tentativi ritenuto congruo (e in questo ultimo caso prendo il risultato migliore)

EPnP

UNIVERSITÀ DI PARMA

- An accurate solution to the PnP problem
  - E ⟶ efficient
  - Non iterative
  - O(n) wrt O(n^x) with $5 \leq x \leq 8$
  - Slightly less precise than iterative methods
  - ICCV 2007

Metodo relativamente nuovo e molto veloce. Non do dettagli in quanto non lo conosco…
Molto rapido ma leggermente meno preciso (e visto che di solito non devo calibrare in real time…)

OpenCV

UNIVERSITÀ DI PARMA

- cv::solvePnP(), different methods:
  - cv::SOLVEPNP_P3P or cv::SOLVEPNP_AP3P: 4 input points
  - cv::SOLVEPNP_IPPE: >= 4 coplanar points
  - cv::SOLVEPNP_IPPE_SQUARE: 4 coplanar points on thje corners of a square (QR code)
- cv::solvePnPRansac(): more than 4 points
- Other refinement methods

Ovviamente ci vuole anche la matrice degli intrinseci

UNIVERSITÀ DI PARMA

Camera Models (2)

Question time!

CAMERA MODELS (2)